LAPORAN TUGAS KELOMPOK
MATA KULIAH PEMROGRAMAN BERORIENTASI OBJEK

SISTEM MANAJEMEN DATA PASIEN PENYAKIT MATA MENGGUNAKAN KONSEP *CLASS* DAN *OBJECT*, ENKAPSULASI, DAN *INHERITANCE* PADA BAHASA PEMROGRAMAN PYTHON



Disusun oleh:

KELOMPOK 3

No	Nama	NIM
1.	Riswan	[NIM]
2.	Yus Aidil	[NIM]
3.	Fahmi	[NIM]
4.	Diva Nurislamia Agfa	[NIM]
5.	Rosa Ramadani	[NIM]
6.	Sofiang	[NIM]
7.	Revalinda	[NIM]
8.	Andi Rezky Nabila	[NIM]
9.	Alfira Yunita Syam	[NIM]

Dosen Pengampu: [Nama Dosen Pengampu]

PROGRAM STUDI [NAMA PROGRAM STUDI]
FAKULTAS [NAMA FAKULTAS]
[NAMA UNIVERSITAS]
2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya kami dapat menyelesaikan laporan tugas kelompok yang berjudul "Sistem Manajemen Data Pasien Penyakit Mata Menggunakan Konsep *Class* dan *Object*, Enkapsulasi, dan *Inheritance* pada Bahasa Pemrograman Python". Laporan ini disusun untuk memenuhi tugas mata kuliah Pemrograman Berorientasi Objek.

Laporan ini membahas perancangan dan implementasi program pengelolaan data pasien penyakit mata yang menggunakan data nyata dari dataset ODIR-5K (Ocular Disease Intelligent Recognition), yaitu 6.392 baris data foto fundus milik 3.358 pasien. Program tersebut menerapkan tiga konsep utama pemrograman berorientasi objek, yaitu *class* dan *object*, enkapsulasi, serta *inheritance* (pewarisan), dan dilengkapi menu interaktif untuk melihat, mencari, menambah, dan merangkum data pasien.

Kami mengucapkan terima kasih kepada [Nama Dosen Pengampu] selaku dosen pengampu mata kuliah Pemrograman Berorientasi Objek yang telah memberikan bimbingan, serta kepada seluruh anggota Kelompok 3 yang telah bekerja sama dengan baik dalam menyelesaikan tugas ini.

Kami menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi perbaikan di masa mendatang. Semoga laporan ini bermanfaat bagi pembaca dan dapat menambah pemahaman tentang penerapan konsep pemrograman berorientasi objek.

[Kota], November 2026

Penyusun,

Kelompok 3

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
DAFTAR TABEL iv
DAFTAR GAMBAR v
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Tujuan 2
1.4 Manfaat 2
1.5 Batasan Masalah 2
BAB II LANDASAN TEORI 4
2.1 Pemrograman Berorientasi Objek 4
2.2 Class dan Object 4
2.3 Enkapsulasi 4
2.4 Inheritance (Pewarisan) 4
2.5 Polimorfisme 5
2.6 Komposisi dan Agregasi 5
2.7 Bahasa Pemrograman Python 5
2.8 Berkas CSV dan Modul csv 5
2.9 Penanganan Exception 6
2.10 Penyakit Mata pada Foto Fundus 6
BAB III PEMBAHASAN 7
3.1 Deskripsi Dataset 7
3.2 Perancangan Sistem 8
3.2.1 Diagram Class 8
3.2.2 Pemetaan Kolom Data ke Atribut 10
3.2.3 Struktur Berkas Program 11
3.2.4 Alur Program 11
3.3 Implementasi Class dan Object 12
3.4 Implementasi Enkapsulasi 13
3.5 Implementasi Inheritance 15

3.5.1 Class Pasien 15
3.5.2 Class PasienMata 15
3.5.3 Hierarki Class Penyakit Mata 16
3.5.4 Polimorfisme 17
3.6 Implementasi Komposisi 18
3.7 Implementasi Program Utama 19
3.8 Implementasi Menu Interaktif 20
3.9 Hasil dan Pengujian 21
3.9.1 Cara Menjalankan Program 21
3.9.2 Keluaran Program 22
3.9.3 Pengujian Program 24
3.10 Pembagian Tugas Anggota Kelompok 26
BAB IV PENUTUP 28
4.1 Kesimpulan 28
4.2 Saran 28
DAFTAR PUSTAKA 30
LAMPIRAN 31
Lampiran A. Kode Program Lengkap 31
A.1 orang.py 31
A.2 pasien.py 32
A.3 pasien_mata.py 32
A.4 pemeriksaan_mata.py 34
A.5 penyakit_mata.py 36
A.6 retinopati_diabetik.py 38
A.7 degenerasi_makula.py 39
A.8 glaukoma.py 40
A.9 kelainan_lain.py 40
A.10 main.py 41
Lampiran B. Keluaran Lengkap Program 47

DAFTAR TABEL

Tabel 3.1 Kolom Dataset yang Digunakan 7
Tabel 3.2 Sebaran Kelompok Penyakit pada Data 7
Tabel 3.3 Pemetaan Kolom Data ke Atribut 10
Tabel 3.4 Struktur Berkas Program 11
Tabel 3.5 Daftar Setter dan Aturan Validasi 14
Tabel 3.6 Perilaku Khusus Class Turunan PenyakitMata 16
Tabel 3.7 Hasil Pengujian Program 24
Tabel 3.8 Pembagian Tugas Anggota Kelompok 26

DAFTAR GAMBAR

Gambar 3.1 Diagram Class Pasien dan Pemeriksaan Mata..... 9
Gambar 3.2 Diagram Class Penyakit Mata 10
Gambar 3.3 Flowchart Program Utama..... 12

BAB I PENDAHULUAN

1.1 Latar Belakang

Pemeriksaan mata di rumah sakit menghasilkan data yang cukup banyak. Salah satu pemeriksaan yang umum adalah foto fundus, yaitu foto bagian dalam belakang bola mata yang memperlihatkan retina, makula, dan saraf optik. Dari foto tersebut, dokter menuliskan diagnosis berupa kata kunci, misalnya *moderate non proliferative retinopathy* atau *cataract*, dan memberi penanda kelompok penyakit untuk setiap pasien.

Dataset ODIR-5K (Ocular Disease Intelligent Recognition) merupakan contoh data nyata tersebut. Dataset ini berisi 6.392 baris data foto fundus dari 3.358 pasien yang dikumpulkan Shanggong Medical Technology dari beberapa rumah sakit di Tiongkok dan dipakai pada kompetisi ODIR-2019 Peking University. Satu pasien bisa memiliki lebih dari satu penyakit; pada data ini terdapat 556 pasien dengan lebih dari satu kelompok penyakit. Data seperti ini rawan salah input, misalnya umur yang tidak masuk akal atau penanda penyakit yang tidak sesuai dengan diagnosisnya, sehingga perlu dikelola secara terstruktur dan diperiksa kebenarannya.

Pemrograman berorientasi objek (*Object-Oriented Programming*, OOP) cocok untuk masalah ini karena data pasien, pemeriksaan mata, dan jenis penyakit dapat dimodelkan sebagai *object* yang memiliki data (atribut) dan perilaku (*method*). Dengan enkapsulasi, setiap nilai diperiksa sebelum disimpan. Dengan *inheritance*, kode yang sama dapat digunakan kembali, misalnya data umur dan jenis kelamin yang diwarisi oleh semua jenis pasien, serta perilaku dasar penyakit yang diwarisi oleh jenis-jenis penyakit yang lebih khusus.

Berdasarkan hal tersebut, kelompok kami membuat program Sistem Manajemen Data Pasien Penyakit Mata menggunakan bahasa pemrograman Python dengan menerapkan konsep *class* dan *object*, enkapsulasi, dan *inheritance*, serta memakai data nyata, bukan data karangan.

1.2 Rumusan Masalah

1. Bagaimana merancang *class* dan *object* untuk mengelola data pasien penyakit mata dari dataset ODIR-5K?
2. Bagaimana menerapkan enkapsulasi agar data pasien, pemeriksaan mata, dan penyakit tetap valid?
3. Bagaimana menerapkan *inheritance* untuk pewarisan bertingkat data pasien dan hierarki jenis penyakit mata?
4. Bagaimana program menangani data yang tidak valid dan menyediakan menu untuk melihat, mencari, menambah, serta merangkum data pasien?

1.3 Tujuan

1. Merancang dan membuat *class* serta *object* untuk data pasien penyakit mata berdasarkan kolom data asli.
2. Menerapkan enkapsulasi dengan atribut *private*, *property*, dan *setter* yang memvalidasi data.
3. Menerapkan *inheritance* bertingkat (`Orang` → `Pasien` → `PasienMata`) dan hierarki *class* penyakit (`PenyakitMata` beserta empat *class* turunannya).
4. Membuat program yang membaca berkas CSV dengan aman, melewati dan mencatat baris yang tidak valid, serta menyediakan menu interaktif.

1.4 Manfaat

1. Bagi mahasiswa: memahami penerapan konsep OOP pada masalah nyata, bukan sekadar contoh sederhana.
2. Bagi pembaca: memperoleh contoh program Python yang terstruktur untuk mengelola dan memvalidasi data.
3. Bagi pengembangan selanjutnya: struktur *class* dapat dikembangkan untuk jenis penyakit atau poli lain.

1.5 Batasan Masalah

1. Program dibuat dengan Python 3 dan hanya menggunakan modul bawaan (`csv` dan `os`).

2. Data yang digunakan adalah berkas `full_df.csv` dari dataset ODIR-5K (Ocular Disease Intelligent Recognition) (6.392 baris, 3.358 pasien). Kolom yang dipakai adalah ID, umur, jenis kelamin, nama foto dan diagnosis kedua mata, serta delapan kolom penanda diagnosis.
3. Penyakit yang dikelola hanya yang terlihat pada foto fundus dan tercatat pada data, yaitu enam kelompok utama (retinopati diabetik, glaukoma, katarak, degenerasi makula, retinopati hipertensi, dan miopia patologis) serta kelainan lain.
4. Program mengelola data diagnosis dari dokter dan tidak melakukan diagnosis dari gambar.
5. Pasien baru hanya disimpan di memori (*list*) selama program berjalan; berkas CSV asli tidak diubah.
6. Program berjalan di terminal (antarmuka teks), tanpa antarmuka grafis dan tanpa basis data.

BAB II LANDASAN TEORI

2.1 Pemrograman Berorientasi Objek

Pemrograman berorientasi objek adalah paradigma pemrograman yang menyusun program sebagai kumpulan *object* yang saling berinteraksi. Setiap *object* menggabungkan data (atribut) dan perilaku (*method*) dalam satu kesatuan. Pilar utama OOP antara lain *class* dan *object*, enkapsulasi, *inheritance*, dan polimorfisme (Lott & Phillips, 2021).

2.2 Class dan Object

Class adalah cetakan (*blueprint*) yang mendefinisikan atribut dan *method*, sedangkan *object* adalah wujud nyata (*instance*) dari *class* dengan nilai atributnya sendiri. Di Python, *class* didefinisikan dengan kata kunci `class`, dan *object* dibuat dengan memanggil nama *class* seperti fungsi. Method khusus `__init__` (*constructor*) dijalankan otomatis saat *object* dibuat, sedangkan parameter `self` merujuk pada *object* yang sedang digunakan (Python Software Foundation, n.d.-c).

2.3 Enkapsulasi

Enkapsulasi adalah pembungkusan data di dalam *object* dan pembatasan akses langsung terhadap data tersebut. Di Python, atribut yang namanya diawali dua garis bawah (misalnya `__umur`) mengalami *name mangling*, yaitu namanya diubah menjadi `_NamaClass__umur`, sehingga tidak dapat diakses langsung dengan nama aslinya dari luar *class* (Python Software Foundation, n.d.-c). Akses dilakukan melalui *property*: dekorator `@property` membuat *getter*, dan `@nama.setter` membuat *setter* yang dapat memvalidasi nilai sebelum disimpan (Python Software Foundation, n.d.-b).

2.4 Inheritance (Pewarisan)

Inheritance memungkinkan sebuah *class* turunan (*child class*) mewarisi atribut dan *method* dari *class* induk (*parent class*). Pewarisan bertingkat (*multilevel inheritance*) terjadi ketika sebuah *class* turunan menjadi induk bagi *class* lain, misalnya A → B → C. Fungsi `super()` digunakan untuk memanggil *method* milik

induk, termasuk *constructor* (`super().__init__()`), yang disebut *constructor chaining*. *Method overriding* adalah penulisan ulang *method* induk di *class* turunan dengan nama yang sama untuk menyesuaikan perilakunya.

2.5 Polimorfisme

Polimorfisme berarti satu perintah yang sama dapat menghasilkan perilaku berbeda bergantung pada *class object* yang menerimanya. Di Python, ketika `objek.method()` dipanggil, Python mencari *method* mulai dari *class object* tersebut, lalu ke induknya. Karena itu, *object* dari *class* turunan yang meng-*override method* akan menjalankan versinya sendiri.

2.6 Komposisi dan Agregasi

Komposisi adalah hubungan "memiliki" (*has-a*) ketika sebuah *object* tersusun dari *object* lain yang menjadi bagiannya, misalnya pasien memiliki data pemeriksaan mata kiri dan kanan. Agregasi juga hubungan "memiliki", tetapi *object* yang dirujuk dapat dipakai bersama oleh banyak *object* lain. Hubungan ini berbeda dengan *inheritance* yang bermakna "adalah" (*is-a*).

2.7 Bahasa Pemrograman Python

Python adalah bahasa pemrograman tingkat tinggi yang mudah dibaca dan mendukung paradigma berorientasi objek. Python menyediakan banyak modul bawaan (*standard library*), seperti `csv` untuk membaca berkas CSV dan `os` untuk bekerja dengan lokasi berkas, sehingga program dapat dijalankan tanpa instalasi tambahan (Matthes, 2023).

2.8 Berkas CSV dan Modul csv

CSV (*Comma-Separated Values*) adalah format berkas teks yang menyimpan data tabel, dengan setiap baris sebagai satu data dan kolom dipisahkan tanda koma. Modul `csv` menyediakan `csv.DictReader` yang membaca setiap baris sebagai *dictionary* dengan nama kolom dari baris pertama (*header*) sebagai kunci. Parameter `restval` menentukan nilai pengganti untuk kolom yang hilang (Python Software Foundation, n.d.-d).

2.9 Penanganan Exception

Exception adalah kejadian kesalahan saat program berjalan. Blok `try` berisi kode yang mungkin menimbulkan kesalahan, sedangkan blok `except` menangkap kesalahan tersebut sehingga program tidak berhenti. `ValueError` adalah *exception* bawaan yang menandakan nilai tidak sesuai, dan dapat dimunculkan sendiri dengan perintah `raise` (Python Software Foundation, n.d.-a).

2.10 Penyakit Mata pada Foto Fundus

Foto fundus memperlihatkan bagian belakang bola mata. Beberapa penyakit yang dapat terlihat pada foto ini dan tercatat pada dataset ODIR-5K adalah retinopati diabetik (kerusakan pembuluh darah retina akibat diabetes), glaukoma (kerusakan saraf optik), katarak (kekeruhan lensa mata), degenerasi makula terkait usia, retinopati hipertensi (kerusakan retina akibat tekanan darah tinggi), dan miopia patologis. Gangguan penglihatan merupakan masalah kesehatan global yang dialami sedikitnya 2,2 miliar orang (World Health Organization, 2019), sehingga pengelolaan data pemeriksaan mata yang rapi menjadi penting.

BAB III PEMBAHASAN

3.1 Deskripsi Dataset

Data yang digunakan adalah berkas `full_df.csv` dari dataset ODIR-5K (Ocular Disease Intelligent Recognition) yang diunduh dari Kaggle (https://www.kaggle.com/datasets/andrewmvd/ocular-disease-recognition-odir5k). Berkas ini berisi 6.392 baris dan 19 kolom. Setiap baris mewakili satu foto fundus, sehingga pasien yang kedua fotonya tersedia muncul pada dua baris dengan data pasien yang sama. Setelah digabung per pasien, terdapat 3.358 pasien: 1.800 laki-laki dan 1.558 perempuan, berumur 1–91 tahun dengan rata-rata 57,9 tahun. Kolom yang digunakan ditunjukkan pada Tabel 3.1.

Tabel 3.1 Kolom Dataset yang Digunakan

Kolom	Keterangan	Contoh
`ID`	nomor pasien	0
`Patient Age`	umur pasien (tahun)	69
`Patient Sex`	jenis kelamin	Female
`Left-Fundus`, `Right-Fundus`	nama file foto mata kiri dan kanan	0_left.jpg
`Left-Diagnostic Keywords`, `Right-Diagnostic Keywords`	kata kunci diagnosis dokter untuk setiap mata	cataract
`N`, `D`, `G`, `C`, `A`, `H`, `M`, `O`	penanda kelompok diagnosis (1 = ada, 0 = tidak)	0 0 0 1 0 0 0 0

Arti penanda: N = normal, D = retinopati diabetik, G = glaukoma, C = katarak, A = degenerasi makula, H = retinopati hipertensi, M = miopia patologis, O = kelainan lain. Kolom `filepath`, `labels`, `target`, dan `filename` tidak digunakan karena berisi label per foto yang diturunkan dari kolom lain, bukan data pasien. Sebaran kelompok penyakit hasil olahan program ditunjukkan pada Tabel 3.2.

Tabel 3.2 Sebaran Kelompok Penyakit pada Data

Kode	Penyakit	Bagian Mata	Pasien	Rata-rata Umur	Kedua Mata
D	Retinopati diabetik	retina	1.105	56,4	659
G	Glaukoma	saraf optik	206	62,6	110
C	Katarak	lensa mata	208	66,4	99
A	Degenerasi makula	makula (pusat retina)	163	60,9	116
H	Retinopati hipertensi	retina	103	56,2	90
M	Miopia patologis	retina	171	57,0	91

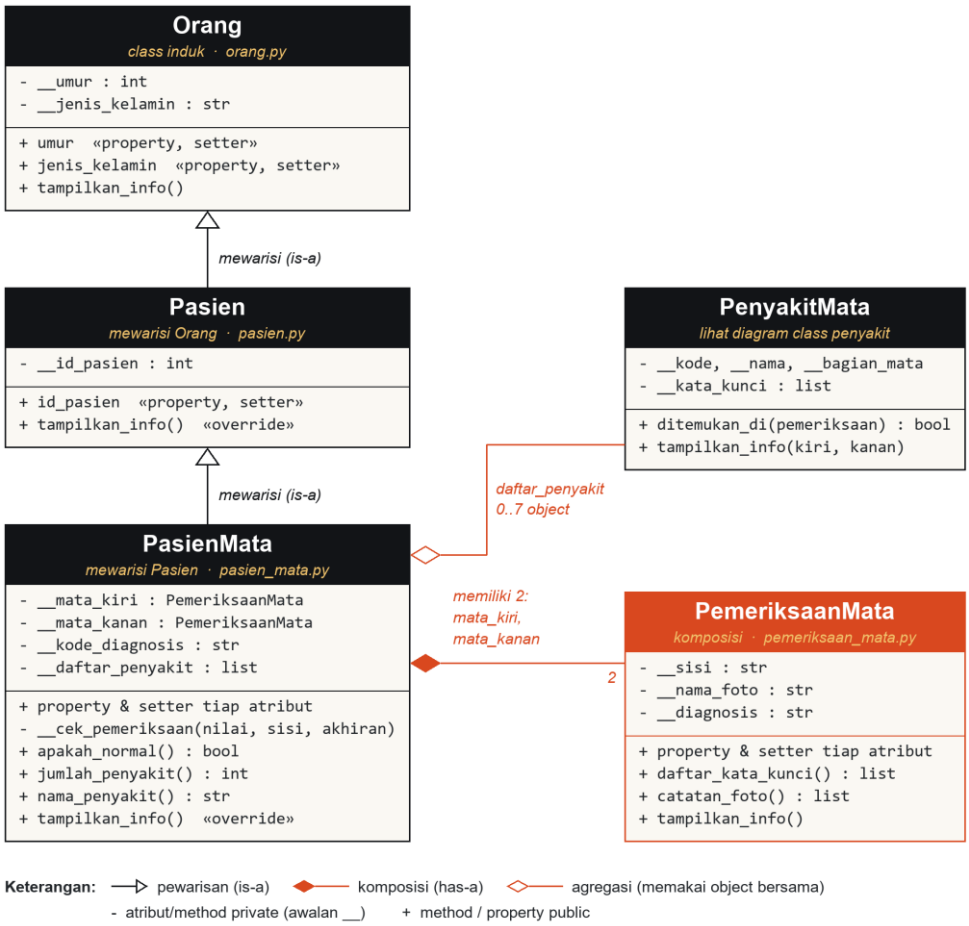
Kode	Penyakit	Bagian Mata	Pasien	Rata-rata Umur	Kedua Mata
O	Kelainan lain	bervariasi	905	59,1	173

Sebanyak 1.080 pasien normal dan 556 pasien memiliki lebih dari satu kelompok penyakit, sehingga jumlah pada Tabel 3.2 lebih besar daripada jumlah pasien. Beberapa catatan lain dari data asli: 567 pasien memakai koma lebar (,) sebagai pemisah diagnosis, terdapat catatan kualitas foto seperti *lens dust* pada 225 foto, dan terdapat 15 pasien berumur 1 tahun yang kemungkinan merupakan kesalahan pencatatan pada sumber data. Data tidak diubah sedikit pun; seluruh pemeriksaan dilakukan oleh program.

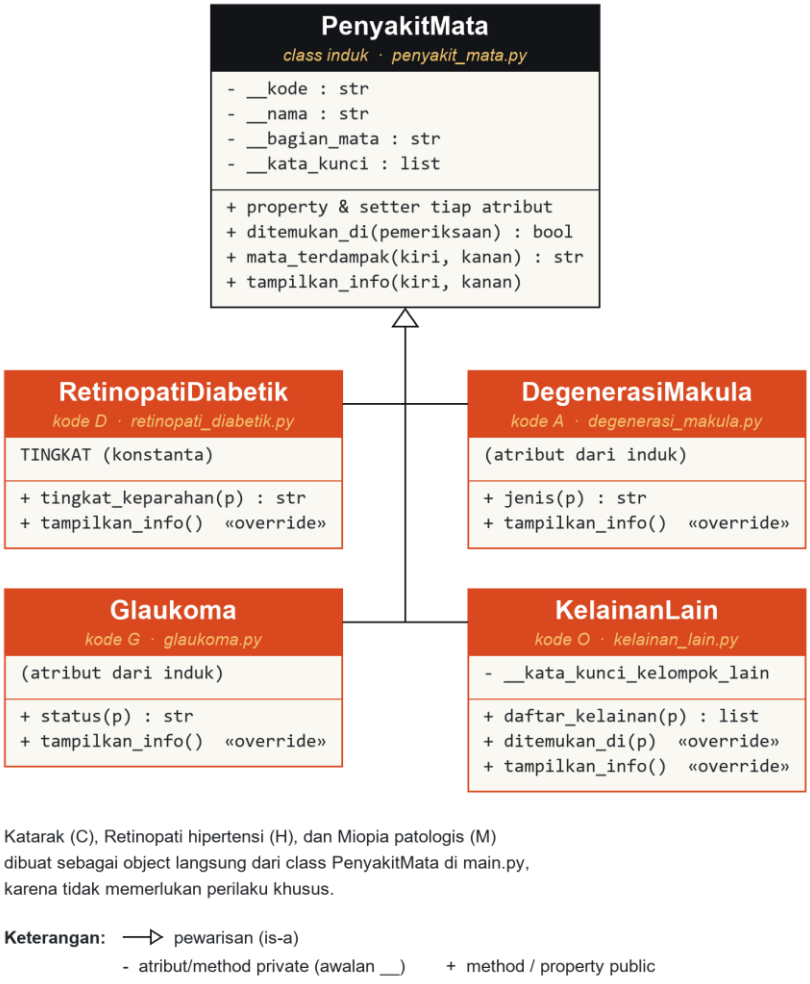
3.2 Perancangan Sistem

3.2.1 Diagram Class

Program terdiri atas sembilan *class*. Gambar 3.1 menunjukkan pewarisan bertingkat `Orang` → `Pasien` → `PasienMata` serta komposisi `PasienMata` dengan dua *object* `PemeriksaanMata`. Gambar 3.2 menunjukkan hierarki *class* penyakit mata.



Gambar 3.1 Diagram Class Pasien dan Pemeriksaan Mata



Gambar 3.2 Diagram Class Penyakit Mata

3.2.2 Pemetaan Kolom Data ke Atribut

Setiap kolom yang digunakan dipetakan ke atribut *class* tanpa menambahkan atribut di luar data, seperti ditunjukkan pada Tabel 3.3.

Tabel 3.3 Pemetaan Kolom Data ke Atribut

Kolom CSV	Class	Atribut	Aturan Validasi
`ID`	Pasien	`id_pasien`	bilangan bulat ≥ 0
`Patient Age`	Orang	`umur`	bilangan bulat 1–120
`Patient Sex`	Orang	`jenis_kelamin`	Male / Female
`Left-Fundus, Right-Fundus`	PemeriksaanMata	`nama_foto`	akhiran _left.jpg / _right.jpg, sesuai ID
`Left- / Right-Diagnostic Keywords`	PemeriksaanMata	`diagnosis`	tidak boleh kosong
`N D G C A H M O`	PasienMata	`kode_diagnosis`	hanya 0/1; N tidak boleh bersama penyakit

Kolom CSV	Class	Atribut	Aturan Validasi
(dikenali dari kata kunci)	PasienMata	`daftar_penyakit`	harus cocok dengan penanda dokter

3.2.3 Struktur Berkas Program

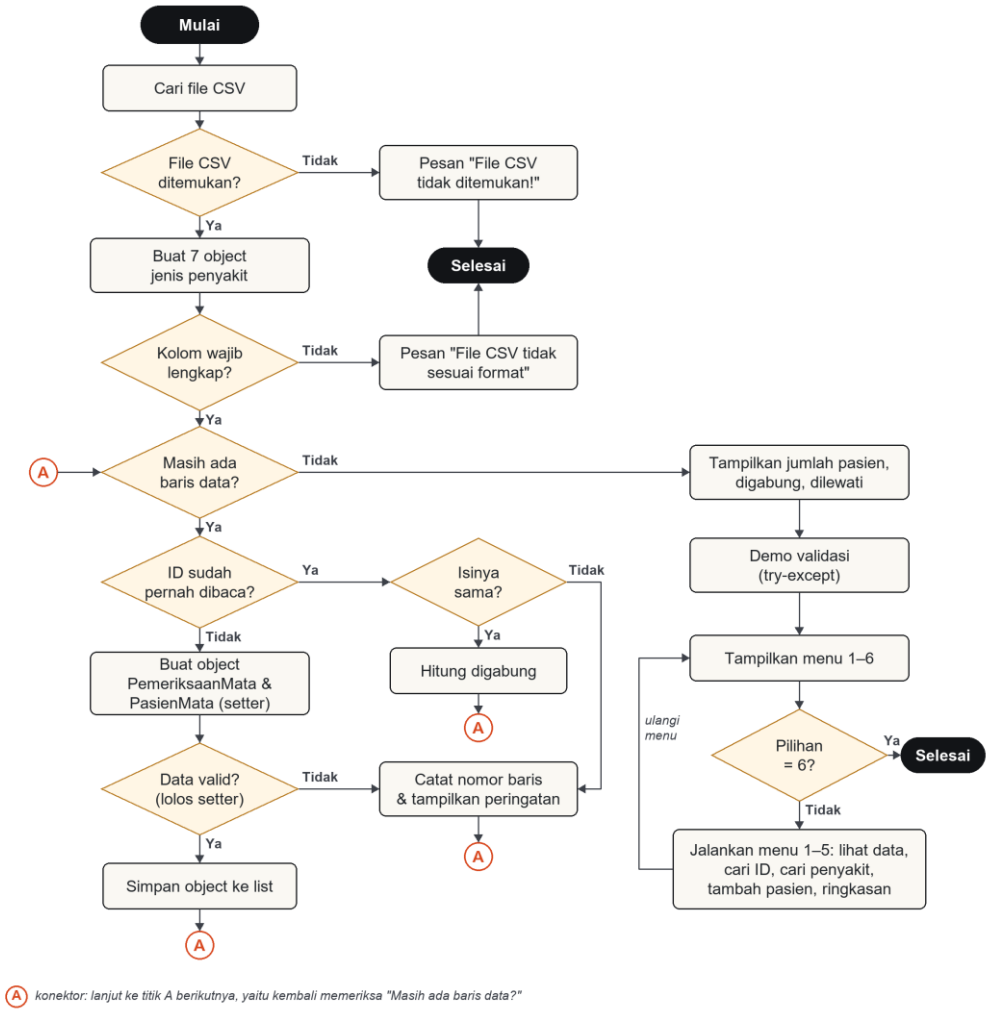
Setiap *class* ditempatkan pada berkas tersendiri dengan penamaan *snake_case*, ditambah `main.py` sebagai program utama (Tabel 3.4).

Tabel 3.4 Struktur Berkas Program

Berkas	Isi
`main.py`	Program utama: membaca CSV, demo validasi, menu interaktif
`orang.py`	class Orang: umur, jenis_kelamin
`pasien.py`	class Pasien(Orang): id_pasien
`pasien_mata.py`	class PasienMata(Pasien): mata kiri/kanan, kode diagnosis, daftar penyakit
`pemeriksaan_mata.py`	class PemeriksaanMata: hasil foto satu mata
`penyakit_mata.py`	class PenyakitMata: satu jenis penyakit mata
`retinopati_diabetik.py`	class RetinopatiDiabetik(PenyakitMata)
`degenerasi_makula.py`	class DegenerasiMakula(PenyakitMata)
`glaukoma.py`	class Glaukoma(PenyakitMata)
`kelainan_lain.py`	class KelainanLain(PenyakitMata)
`data/full_df.csv`	data asli ODIR-5K (6.392 baris)

3.2.4 Alur Program

Alur program utama ditunjukkan pada Gambar 3.3. Program mencari berkas CSV, membuat tujuh *object* jenis penyakit, membaca setiap baris, menggabungkan baris milik pasien yang sama, melewati baris yang tidak valid, lalu menampilkan demo validasi dan menu interaktif.



Gambar 3.3 Flowchart Program Utama

3.3 Implementasi Class dan Object

Class `Orang` merupakan *class* induk paling atas. *Constructor* `__init__` (baris 12) mengisi umur dan jenis kelamin melalui *setter*, sehingga nilainya langsung divalidasi.

orang.py (baris 6–16)

```
 6  class Orang:
 7      """Menyimpan data dasar seseorang: umur dan jenis kelamin."""
 8
 9      # Nilai jenis kelamin yang ada di data (dicek dari nilai unik kolom
    Patient Sex)
10      JENIS_KELAMIN_VALID = ("Male", "Female")
11
12      def __init__(self, umur, jenis_kelamin):
13          # [ENKAPSULASI] Nilai diisi lewat setter, jadi langsung divalidasi
14          # sebelum disimpan ke atribut private (self.__umur,
    self.__jenis_kelamin).
15          self.umur = umur
16          self.jenis_kelamin = jenis_kelamin
```

Object dibuat di fungsi `buat_pasien()` pada `main.py`. Satu baris CSV menghasilkan dua *object* `PemeriksaanMata` dan satu *object* `PasienMata` (baris 93–106). Setiap *object* yang berhasil dibuat disimpan ke *list* `daftar_pasien` dengan `append` (baris 132). Saat program berjalan terbentuk 3.358 *object* `PasienMata`, 6.716 *object* `PemeriksaanMata`, dan 7 *object* jenis penyakit.

main.py (baris 93–106)

```
 93  def buat_pasien(baris, daftar_jenis):
 94      """[CLASS & OBJECT] Mengubah satu baris CSV menjadi object PasienMata."""
 95      # [KOMPOSISI] Object PemeriksaanMata dibuat dulu, lalu dimasukkan ke
     PasienMata
 96      mata_kiri = PemeriksaanMata("kiri", baris["Left-Fundus"].strip(),
     baris["Left-Diagnostic Keywords"])
 97      mata_kanan = PemeriksaanMata("kanan", baris["Right-Fundus"].strip(),
     baris["Right-Diagnostic Keywords"])
 98      return PasienMata(
 99          ke_bilangan_bulat(baris["ID"], "ID"),
100          ke_bilangan_bulat(baris["Patient Age"], "Umur"),
101          baris["Patient Sex"].strip(),
102          mata_kiri,
103          mata_kanan,
104          baca_kode_diagnosis(baris),
105          deteksi_penyakit(mata_kiri, mata_kanan, daftar_jenis),
106      )
```

3.4 Implementasi Enkapsulasi

Seluruh atribut dibuat *private* dengan awalan dua garis bawah dan diakses melalui *property*. Contoh pada atribut umur: *getter* `@property` (baris 19) mengembalikan `self.__umur`, sedangkan *setter* (baris 23) menolak nilai yang bukan bilangan bulat 1–120 dengan `raise ValueError`.

orang.py (baris 19–28)

```
19      @property
20      def umur(self):
21          return self.__umur
22
23      @umur.setter
24      def umur(self, nilai):
25          # Aturan: umur bilangan bulat 1 sampai 120 tahun
26          if not isinstance(nilai, int) or nilai < 1 or nilai > 120:
27              raise ValueError(f"Umur harus bilangan bulat 1 sampai 120, bukan
    {nilai}")
28          self.__umur = nilai
```

Validasi terpenting terdapat pada *setter* `daftar_penyakit` di `PasienMata`. *Setter* ini melakukan validasi silang: kode penyakit yang dikenali dari kata kunci diagnosis harus sama dengan penanda yang diberikan dokter. Pengujian pada seluruh data menunjukkan 3.358 pasien cocok 100%. *Getter*

`daftar_penyakit` (baris 73) mengembalikan salinan *list*, sehingga isi asli tidak dapat diubah dari luar.

pasien_mata.py (baris 75–90)

```
75      @daftar_penyakit.setter
76      def daftar_penyakit(self, nilai):
77          if not isinstance(nilai, list):
78              raise ValueError("Daftar penyakit harus berupa list")
79          kode_ditemukan = ""
80          for penyakit in nilai:
81              if not isinstance(penyakit, PenyakitMata):
82                  raise ValueError("Isi daftar penyakit harus object
    PenyakitMata")
83              kode_ditemukan += penyakit.kode
84          # Validasi silang: penyakit yang dikenali dari kata kunci diagnosis
    harus sama
85          # dengan penanda yang diberi dokter (kolom D, G, C, A, H, M, O).
86          kode_penanda = self.kode_diagnosis.replace("N", "")
87          if sorted(kode_ditemukan) != sorted(kode_penanda):
88              raise ValueError(f"Penanda diagnosis '{self.kode_diagnosis}'
    tidak cocok dengan "
89                               f"kata kunci diagnosis (terbaca:
    '{kode_ditemukan or 'N'}')")
90          self.__daftar_penyakit = list(nilai)
```

Secara keseluruhan terdapat 15 *setter* dengan aturan validasi seperti pada Tabel 3.5.

Tabel 3.5 Daftar Setter dan Aturan Validasi

Berkas	Baris	Atribut	Aturan
orang.py	23	umur	bilangan bulat 1–120
orang.py	34	jenis_kelamin	Male atau Female
pasien.py	22	id_pasien	bilangan bulat ≥ 0
pasien_mata.py	33	mata_kiri	object PemeriksaanMata sisi kiri, nama foto sesuai ID
pasien_mata.py	41	mata_kanan	object PemeriksaanMata sisi kanan, nama foto sesuai ID
pasien_mata.py	58	kode_diagnosis	huruf N D G C A H M O, tidak ganda, N tidak bersama penyakit
pasien_mata.py	75	daftar_penyakit	list object PenyakitMata yang cocok dengan penanda dokter
pemeriksaan_mata.py	30	sisi	kiri atau kanan
pemeriksaan_mata.py	40	nama_foto	berakhiran _left.jpg atau _right.jpg sesuai sisi
pemeriksaan_mata.py	52	diagnosis	teks tidak kosong
penyakit_mata.py	26	kode	salah satu dari D, G, C, A, H, M, O
penyakit_mata.py	36	nama	teks tidak kosong
penyakit_mata.py	46	bagian_mata	teks tidak kosong
penyakit_mata.py	56	kata_kunci	list berisi teks yang tidak kosong
kelainan_lain.py	26	kata_kunci_kelompok_lain	list yang tidak kosong

3.5 Implementasi Inheritance

3.5.1 Class Pasien

`Pasien` mewarisi `Orang` (baris 8). *Constructor*-nya memanggil `super().__init__()` (baris 13) sehingga umur dan jenis kelamin tetap diurus `Orang`. *Method* `tampilkan_info()` di-*override* dan memanggil versi induknya dengan `super()`.

pasien.py (baris 8–33)

```
 8  class Pasien(Orang):
 9      """Pasien rumah sakit (turunan dari Orang)."""
10
11      def __init__(self, id_pasien, umur, jenis_kelamin):
12          # [INHERITANCE] Constructor chaining: umur & jenis kelamin diurus
    oleh Orang
13          super().__init__(umur, jenis_kelamin)
14          # [ENKAPSULASI] Diisi lewat setter agar tervalidasi
15          self.id_pasien = id_pasien
16
17      # ---------- [ENKAPSULASI] getter & setter ----------
18      @property
19      def id_pasien(self):
20          return self.__id_pasien
21
22      @id_pasien.setter
23      def id_pasien(self, nilai):
24          # Aturan: bilangan bulat 0 atau lebih (di data memang ada pasien
    dengan ID 0)
25          if not isinstance(nilai, int) or nilai < 0:
26              raise ValueError(f"ID pasien harus bilangan bulat 0 atau lebih,
    bukan {nilai}")
27          self.__id_pasien = nilai
28
29      # ---------- [INHERITANCE] Method overriding ----------
30      def tampilkan_info(self):
31          """Override tampilkan_info() milik Orang: tambah ID pasien."""
32          print(f"ID Pasien        : {self.id_pasien}")
33          super().tampilkan_info()  # memakai ulang tampilan umur & jenis
    kelamin dari Orang
```

3.5.2 Class PasienMata

`PasienMata` mewarisi `Pasien` (baris 12) sehingga terbentuk pewarisan bertingkat `Orang` → `Pasien` → `PasienMata`. *Constructor*-nya meneruskan ID, umur, dan jenis kelamin ke `Pasien` dengan `super().__init__()` (baris 17–26).

pasien_mata.py (baris 17–26)

```
17      def __init__(self, id_pasien, umur, jenis_kelamin, mata_kiri, mata_kanan,
18                   kode_diagnosis, daftar_penyakit):
19          # [INHERITANCE] Constructor chaining: Pasien -> Orang
20          super().__init__(id_pasien, umur, jenis_kelamin)
21          # [ENKAPSULASI] Urutan penting: kode_diagnosis diisi sebelum
    daftar_penyakit
22          # karena setter daftar_penyakit mencocokkan keduanya.
23          self.mata_kiri = mata_kiri
24          self.mata_kanan = mata_kanan
25          self.kode_diagnosis = kode_diagnosis
26          self.daftar_penyakit = daftar_penyakit
```

Method `tampilkan_info()` (baris 110–121) di-*override* lagi: versi `Pasien` dipanggil lebih dulu, lalu data kedua mata dan daftar penyakit ditampilkan.

pasien_mata.py (baris 110–121)

```
110     def tampilkan_info(self):
111         """Override tampilkan_info() milik Pasien: tambah hasil pemeriksaan
    dan penyakit."""
112         super().tampilkan_info()  # tampilkan ID, umur, jenis kelamin
113         self.mata_kiri.tampilkan_info()
114         self.mata_kanan.tampilkan_info()
115         if self.apakah_normal():
116             print("Penyakit         : tidak ada (normal)")
117         else:
118             print(f"Penyakit         : {self.jumlah_penyakit()} kelompok")
119             for penyakit in self.__daftar_penyakit:
120                 # [POLIMORFISME] Versi tampilkan_info() mengikuti class
    object penyakitnya
121                 penyakit.tampilkan_info(self.mata_kiri, self.mata_kanan)
```

3.5.3 Hierarki Class Penyakit Mata

Class `PenyakitMata` menjadi induk bagi empat *class* turunan yang memiliki perilaku khusus (Tabel 3.6). Katarak, retinopati hipertensi, dan miopia patologis dibuat sebagai *object* langsung dari `PenyakitMata` karena tidak memerlukan perilaku khusus.

Tabel 3.6 Perilaku Khusus Class Turunan PenyakitMata

Class	Method Khusus	Hasil pada Data (mata)
`RetinopatiDiabetik`	`tingkat_keparahan()`	proliferatif 32, berat 158, sedang 977, ringan 543, tidak disebutkan 54
`DegenerasiMakula`	`jenis()`	kering 227, basah 49, tidak disebutkan 3
`Glaukoma`	`status()`	pasti 274, dugaan 42
`KelainanLain`	`daftar_kelainan()`, *override* `ditemukan_di()`	67 jenis kelainan

Contoh implementasinya adalah `RetinopatiDiabetik` (baris 9–42). *Constructor* memanggil `super().__init__()` dengan kode D dan kata kuncinya, *method* `tingkat_keparahan()` mencari tingkat dari yang paling berat, dan `tampilkan_info()` di-*override* untuk menambahkan tingkat setiap mata.

retinopati_diabetik.py (baris 9–42)

```
 9  class RetinopatiDiabetik(PenyakitMata):
10      """Kerusakan pembuluh darah retina akibat diabetes."""
11
12      # Urutan dari yang paling berat: (kata kunci di data, nama tingkat)
13      TINGKAT = (
14          ("proliferative diabetic retinopathy", "proliferatif"),
15          ("severe nonproliferative retinopathy", "berat"),
16          ("moderate non proliferative retinopathy", "sedang"),
17          ("mild nonproliferative retinopathy", "ringan"),
18      )
19
20      def __init__(self):
21          # [INHERITANCE] Constructor chaining dengan nilai khusus retinopati
    diabetik
22          super().__init__("D", "Retinopati diabetik", "retina",
23                           ["nonproliferative retinopathy", "non proliferative
    retinopathy",
24                            "proliferative diabetic retinopathy", "diabetic
    retinopathy"])
25
26      # ---------- Method khusus RetinopatiDiabetik ----------
27      def tingkat_keparahan(self, pemeriksaan):
28          """Tingkat retinopati diabetik di satu mata (dicari dari yang paling
    berat)."""
29          if not self.ditemukan_di(pemeriksaan):
30              return "-"
31          for kunci, tingkat in RetinopatiDiabetik.TINGKAT:
32              for kata in pemeriksaan.daftar_kata_kunci():
33                  if kunci in kata:
34                      return tingkat
35          return "tidak disebutkan"
36
37      # ---------- [INHERITANCE] Method overriding ----------
38      def tampilkan_info(self, mata_kiri, mata_kanan):
39          """Override: tambahkan tingkat keparahan di setiap mata."""
40          super().tampilkan_info(mata_kiri, mata_kanan)
41          print(f"      tingkat: kiri {self.tingkat_keparahan(mata_kiri)}, "
42                f"kanan {self.tingkat_keparahan(mata_kanan)}")
```

3.5.4 Polimorfisme

Polimorfisme terlihat pada `tampilkan_info()` milik `PasienMata` (baris 119–121). Perintah `penyakit.tampilkan_info()` sama untuk setiap penyakit, tetapi versi yang dijalankan mengikuti *class object*-nya: retinopati diabetik menampilkan tingkat, kelainan lain menampilkan jenis, dan katarak memakai versi induk. `KelainanLain` juga meng-*override* `ditemukan_di()` karena kelainan lain dikenali dari kata kunci yang bukan milik enam kelompok lain.

kelainan_lain.py (baris 47–50)

```
47      # ---------- [POLIMORFISME] Method overriding ----------
48      def ditemukan_di(self, pemeriksaan):
49          """Override: kelainan lain ditemukan jika daftar_kelainan() tidak
    kosong."""
50          return len(self.daftar_kelainan(pemeriksaan)) > 0
```

3.6 Implementasi Komposisi

`PasienMata` memiliki dua *object* `PemeriksaanMata` (mata kiri dan kanan) dan *list object* `PenyakitMata`. Hubungan ini adalah komposisi (*has-a*), bukan pewarisan, karena mata bukan pasien. *Method private* `__cek_pemeriksaan()` (baris 45–52) memakai `isinstance` untuk memastikan *object* yang dimasukkan benar dan nama fotonya sesuai ID pasien.

pasien_mata.py (baris 45–52)

```
45      def __cek_pemeriksaan(self, nilai, sisi, akhiran):
46          """Method private: pemeriksaan harus object PemeriksaanMata, sisinya
    benar,
47          dan nama fotonya sesuai ID pasien (contoh: ID 7 -> 7_left.jpg)."""
48          if not isinstance(nilai, PemeriksaanMata) or nilai.sisi != sisi:
49              raise ValueError(f"Mata {sisi} harus berupa object
    PemeriksaanMata sisi {sisi}")
50          if nilai.nama_foto != f"{self.id_pasien}{akhiran}":
51              raise ValueError(f"Nama foto '{nilai.nama_foto}' tidak sesuai
    dengan ID pasien {self.id_pasien}")
52          return nilai
```

`PemeriksaanMata` membaca diagnosis sesuai data asli (baris 60–75): koma lebar diganti koma biasa sebelum diagnosis dipecah menjadi kata kunci, dan catatan kualitas foto seperti *lens dust* dipisahkan agar tidak dihitung sebagai penyakit.

pemeriksaan_mata.py (baris 60–75)

```
60      def daftar_kata_kunci(self):
61          """Memecah diagnosis menjadi daftar kata kunci berhuruf kecil."""
62          teks = self.diagnosis.replace(PemeriksaanMata.KOMA_LEBAR,
    ",").lower()
63          hasil = []
64          for kata in teks.split(","):
65              if kata.strip() != "":
66                  hasil.append(kata.strip())
67          return hasil
68
69      def catatan_foto(self):
70          """Catatan kualitas foto (bukan penyakit), misalnya 'lens dust'."""
71          hasil = []
72          for kata in self.daftar_kata_kunci():
73              if kata in PemeriksaanMata.BUKAN_PENYAKIT and kata != "normal
    fundus":
74                  hasil.append(kata)
75          return hasil
```

3.7 Implementasi Program Utama

Data dibaca oleh fungsi `muat_data()` (baris 109–138). Berkas dibuka dengan `encoding="utf-8-sig"` dan dibaca menggunakan `csv.DictReader(file, restval="")`. Sebelum membaca data, program memeriksa apakah seluruh kolom wajib ada pada *header*. Setiap baris diproses di dalam blok `try`: baris kedua milik pasien yang sama harus berisi data yang sama lalu digabung, sedangkan baris baru diubah menjadi *object* melalui `buat_pasien()`. Jika muncul `ValueError`, nomor baris CSV dicatat (`pembaca.line_num`), peringatan dicetak, dan program tetap berjalan.

main.py (baris 109–138)

```
109 def muat_data(lokasi_file, daftar_jenis):
110     """Membaca CSV, menggabungkan baris milik pasien yang sama, dan mencatat
    baris tidak valid."""
111     daftar_pasien = []
112     isi_per_id = {}        # ID -> isi kolom pasien dari baris pertama yang
    valid
113     baris_dilewati = []    # nomor baris CSV yang tidak valid
114     baris_digabung = 0     # baris foto mata kedua dari pasien yang sama
115     # utf-8-sig: tetap aman jika CSV pernah disimpan ulang lewat Excel (ada
    BOM)
116     with open(lokasi_file, mode="r", encoding="utf-8-sig", newline="") as
    file:
117         # restval="": kolom yang hilang dianggap teks kosong, jadi ikut
    ditolak validasi
118         pembaca = csv.DictReader(file, restval="")
119         for kolom in KOLOM_PASIEN:
120             if kolom not in (pembaca.fieldnames or []):
121                 raise ValueError(f"Kolom '{kolom}' tidak ada di file CSV")
122         for baris in pembaca:
123             try:
124                 isi = tuple(baris[kolom] for kolom in KOLOM_PASIEN)
125                 id_teks = baris["ID"].strip()
126                 if id_teks in isi_per_id:
127                     # Baris foto mata kedua: isinya harus sama dengan baris
    pertama
128                     if isi != isi_per_id[id_teks]:
129                         raise ValueError(f"Data pasien ID {id_teks} berbeda
    dengan baris sebelumnya")
130                     baris_digabung += 1
131                     continue
132                 daftar_pasien.append(buat_pasien(baris, daftar_jenis))
133                 isi_per_id[id_teks] = isi
134             except ValueError as error:
135                 # Baris yang melanggar aturan dilewati dan dicatat, program
    tetap berjalan
136                 baris_dilewati.append(pembaca.line_num)
137                 print(f"[PERINGATAN] Baris {pembaca.line_num} dilewati:
    {error}")
138     return daftar_pasien, baris_dilewati, baris_digabung
```

Dari data asli, program membentuk 3.358 *object* `PasienMata`, menggabungkan 3.034 baris, dan melewati 0 baris. Penanganan baris tidak valid diuji dengan berkas CSV yang sengaja dirusak (Subbab 3.9.3).

3.8 Implementasi Menu Interaktif

Menu ditampilkan berulang oleh fungsi `jalankan_menu()` (baris 292–317) menggunakan perulangan `while True` sampai pengguna memilih 6 (Keluar). Pilihan yang tersedia adalah melihat 10 data pertama, mencari pasien berdasarkan ID, mencari pasien berdasarkan penyakit, menambah pasien baru, dan menampilkan ringkasan.

main.py (baris 292–317)

```
292 def jalankan_menu(daftar_pasien, daftar_jenis):
293     """Menampilkan menu berulang-ulang sampai pengguna memilih Keluar."""
294     while True:
295         cetak_judul("MENU")
296         print(f"1. Lihat {JUMLAH_DITAMPILKAN} data pertama")
297         print("2. Cari pasien berdasarkan ID")
298         print("3. Cari pasien berdasarkan penyakit")
299         print("4. Tambah pasien baru")
300         print("5. Tampilkan ringkasan")
301         print("6. Keluar")
302         pilihan = input("Pilih menu (1-6): ").strip()
303         if pilihan == "1":
304             menu_lihat_data(daftar_pasien)
305         elif pilihan == "2":
306             menu_cari_id(daftar_pasien)
307         elif pilihan == "3":
308             menu_cari_penyakit(daftar_pasien, daftar_jenis)
309         elif pilihan == "4":
310             menu_tambah_pasien(daftar_pasien, daftar_jenis)
311         elif pilihan == "5":
312             menu_ringkasan(daftar_pasien, daftar_jenis)
313         elif pilihan == "6":
314             print("Terima kasih. Program selesai.")
315             break
316         else:
317             print("Pilihan tidak tersedia. Masukkan angka 1 sampai 6.")
```

Fungsi `menu_tambah_pasien()` (baris 220–245) membuat ID otomatis (ID terbesar + 1), membaca masukan pengguna, mendeteksi penyakit dari kata kunci diagnosis, lalu membuat *object* `PasienMata` dengan *setter* yang sama seperti data CSV. Data yang tidak valid ditolak, dan berkas CSV asli tidak diubah.

main.py (baris 220–245)

```
220 def menu_tambah_pasien(daftar_pasien, daftar_jenis):
221     """Menu 4: menambah pasien baru ke list (file CSV asli tidak diubah)."""
222     cetak_judul("TAMBAH PASIEN BARU")
223     # ID dibuat otomatis (ID terbesar + 1) agar tidak ada ID ganda
224     id_baru = max(pasien.id_pasien for pasien in daftar_pasien) + 1
225     print(f"ID pasien baru (otomatis)    : {id_baru}")
226     print("Tulis diagnosis dengan istilah dokter seperti di data, pisahkan
    dengan koma.")
227     print("Contoh: normal fundus / cataract / glaucoma / mild
    nonproliferative retinopathy")
228     try:
229         umur = ke_bilangan_bulat(input("Umur                          : "),
    "Umur")
230         jenis_kelamin = input("Jenis kelamin (Male/Female)  : ").strip()
231         mata_kiri = PemeriksaanMata("kiri", f"{id_baru}_left.jpg",
    input("Diagnosis mata kiri          : "))
232         mata_kanan = PemeriksaanMata("kanan", f"{id_baru}_right.jpg",
    input("Diagnosis mata kanan         : "))
233         daftar_penyakit = deteksi_penyakit(mata_kiri, mata_kanan,
    daftar_jenis)
234         kode = ""
235         for penyakit in daftar_penyakit:
236             kode += penyakit.kode
237         # [ENKAPSULASI] Data baru diperiksa oleh setter yang sama dengan data
    CSV
238         pasien = PasienMata(id_baru, umur, jenis_kelamin, mata_kiri,
    mata_kanan, kode or "N", daftar_penyakit)
239     except ValueError as error:
240         print(f"Data tidak valid: {error}")
241         print("Pasien baru tidak ditambahkan.")
242         return
243     daftar_pasien.append(pasien)
244     print(f"Pasien ID {id_baru} berhasil ditambahkan
    ({pasien.nama_penyakit()}). "
245           f"Jumlah pasien sekarang: {len(daftar_pasien)}")
```

3.9 Hasil dan Pengujian

3.9.1 Cara Menjalankan Program

1. Pastikan Python 3 telah terpasang (periksa dengan perintah `python --version`).
2. Pastikan berkas data berada di `program/data/full_df.csv`.
3. Buka terminal pada folder proyek, masuk ke folder program dengan perintah `cd program`, lalu jalankan `python main.py`.
4. Setelah keluaran awal tampil, ketik angka 1–6 pada menu lalu tekan Enter. Pilih 6 untuk keluar.
5. Program tidak memerlukan instalasi *library* tambahan karena hanya menggunakan modul bawaan Python.

3.9.2 Keluaran Program

Seluruh keluaran pada laporan ini diambil dari program yang benar-benar dijalankan. Bagian awal keluaran menampilkan hasil pemuatan data dan demo enkapsulasi, yaitu data tidak valid yang ditolak *setter*.

```
> python main.py
============================================================
SISTEM MANAJEMEN DATA PASIEN PENYAKIT MATA
============================================================
File data          : full_df.csv
Pasien dimuat      : 3358 object PasienMata
Baris digabung     : 3034 (foto mata kedua dari pasien yang sama)
Baris dilewati     : 0

============================================================
DEMO ENKAPSULASI (data tidak valid ditolak)
============================================================
Umur pasien ID 0 sebelum diubah : 69
Mencoba mengubah: pasien.umur = -5
ValueError tertangkap: Umur harus bilangan bulat 1 sampai 120, bukan -5
Umur pasien ID 0 sesudahnya     : 69 (tidak berubah)

Mencoba membuat pasien berpenanda katarak (C), tetapi diagnosisnya normal
ValueError tertangkap: Penanda diagnosis 'C' tidak cocok dengan kata kunci
diagnosis (terbaca: 'N')
```

Pilihan 1 menampilkan 10 pasien pertama beserta penyakitnya.

```
Pilih menu (1-6): 1

============================================================
10 DATA PERTAMA (dari 3358 pasien)
============================================================
ID    Umur  Kelamin  Penyakit
0     69    Female   Katarak
1     57    Male     Normal
2     42    Male     Retinopati diabetik, Kelainan lain
4     53    Male     Retinopati diabetik, Kelainan lain
5     50    Female   Retinopati diabetik
6     60    Male     Retinopati diabetik, Kelainan lain
7     60    Female   Retinopati diabetik, Kelainan lain
8     59    Male     Normal
9     54    Male     Kelainan lain
10    70    Male     Kelainan lain
```

Pilihan 2 dengan ID 188 menampilkan data lengkap pasien. Baris ID berasal dari `Pasien`, umur dan jenis kelamin dari `Orang`, sedangkan data mata dan penyakit dari `PasienMata`; setiap penyakit ditampilkan sesuai *class*-nya (polimorfisme).

```
Pilih menu (1-6): 2

================================================================
CARI PASIEN BERDASARKAN ID
================================================================
Masukkan ID pasien: 188
ID Pasien        : 188
Umur             : 55 tahun
Jenis Kelamin    : Female
Mata kiri        : laser spot, severe proliferative diabetic retinopathy
Mata kanan       : cataract
Penyakit         : 3 kelompok
  - Retinopati diabetik (retina): mata kiri
      tingkat: kiri proliferatif, kanan -
  - Katarak (lensa mata): mata kanan
  - Kelainan lain (bervariasi): mata kiri
      jenis: laser spot
```

Pilihan 4 menambah pasien baru. Data yang valid diterima dengan ID otomatis, sedangkan data dengan jenis kelamin “Laki-laki” ditolak oleh *setter*.

```
Pilih menu (1-6): 4

================================================================
TAMBAH PASIEN BARU
================================================================
ID pasien baru (otomatis)    : 4785
Tulis diagnosis dengan istilah dokter seperti di data, pisahkan dengan koma.
Contoh: normal fundus / cataract / glaucoma / mild nonproliferative retinopathy
Umur                         : 63
Jenis kelamin (Male/Female)  : Female
Diagnosis mata kiri          : cataract
Diagnosis mata kanan         : moderate non proliferative retinopathy
Pasien ID 4785 berhasil ditambahkan (Retinopati diabetik, Katarak). Jumlah pasien
sekarang: 3359
…
Pilih menu (1-6): 4

================================================================
TAMBAH PASIEN BARU
================================================================
ID pasien baru (otomatis)    : 4786
Tulis diagnosis dengan istilah dokter seperti di data, pisahkan dengan koma.
Contoh: normal fundus / cataract / glaucoma / mild nonproliferative retinopathy
Umur                         : 45
Jenis kelamin (Male/Female)  : Laki-laki
Diagnosis mata kiri          : normal fundus
Diagnosis mata kanan         : normal fundus
Data tidak valid: Jenis kelamin harus 'Male' atau 'Female', bukan 'Laki-laki'
Pasien baru tidak ditambahkan.
```

Pilihan 5 menampilkan ringkasan yang dihitung dari seluruh *object*.

```
Pilih menu (1-6): 5

================================================================
RINGKASAN DATA
================================================================
Jumlah pasien             : 3358
Laki-laki / Perempuan     : 1800 / 1558
Rata-rata umur            : 57.9 tahun
Pasien normal             : 1080
Pasien > 1 kelompok       : 556

Penyakit                Pasien  Rata umur  Kedua mata
Retinopati diabetik       1105       56.4         659
Glaukoma                   206       62.6         110
Katarak                    208       66.4          99
Degenerasi makula          163       60.9         116
Retinopati hipertensi      103       56.2          90
Miopia patologis           171       57.0          91
Kelainan lain              905       59.1         173

Penyakit terbanyak        : Retinopati diabetik (1105 pasien)
```

Berdasarkan ringkasan tersebut, penyakit terbanyak adalah retinopati diabetik (1.105 pasien atau 32,9% dari seluruh pasien). Katarak memiliki rata-rata umur tertinggi, yaitu 66,4 tahun, sedangkan retinopati hipertensi paling sering mengenai kedua mata (90 dari 103 pasien).

3.9.3 Pengujian Program

Pengujian dilakukan dengan metode *black box*, yaitu memeriksa keluaran program berdasarkan masukan tertentu tanpa melihat struktur internalnya. Pengujian mencakup *setter*, berkas CSV yang sengaja dirusak, berkas tanpa kolom wajib, berkas yang tidak ada, dan setiap pilihan menu. Hasilnya ditunjukkan pada Tabel 3.7. Seluruh 34 skenario memberikan hasil yang sesuai dengan yang diharapkan.

Tabel 3.7 Hasil Pengujian Program

No	Skenario	Masukan	Hasil yang Diharapkan	Hasil Pengujian	Status
1	Memuat data asli	`data/full_df.csv`	3358 pasien, 0 baris dilewati	3358 pasien; 3034 baris digabung; 0 dilewati	Berhasil
2	Umur negatif	`pasien.umur = -5`	ValueError, umur tetap 57	ValueError: Umur harus bilangan bulat 1 sampai 120, bukan -5; umur 57	Berhasil
3	Umur di atas batas	`pasien.umur = 150`	ValueError	ValueError: Umur harus bilangan bulat 1 sampai 120, bukan 150	Berhasil
4	Jenis kelamin tidak valid	`pasien.jenis_kelamin = "Laki-laki"`	ValueError	ValueError: Jenis kelamin harus 'Male' atau 'Female', bukan 'Laki-laki'	Berhasil

No	Skenario	Masukan	Hasil yang Diharapkan	Hasil Pengujian	Status
5	ID negatif	`pasien.id_pasien = -1`	ValueError	ValueError: ID pasien harus bilangan bulat 0 atau lebih, bukan -1	Berhasil
6	Sisi mata tidak valid	`PemeriksaanMata( "atas", ...)`	ValueError	ValueError: Sisi mata harus 'kiri' atau 'kanan', bukan 'atas'	Berhasil
7	Nama foto salah sisi	`foto kiri "1_right.jpg"`	ValueError	ValueError: Nama foto mata kiri harus berakhiran _left.jpg, bukan '1_right.jpg'	Berhasil
8	Diagnosis kosong	`diagnosis "  "`	ValueError	ValueError: Diagnosis mata kiri tidak boleh kosong	Berhasil
9	Normal bersamaan penyakit	`kode_diagnosis = "NC"`	ValueError	ValueError: Penanda normal (N) tidak boleh bersamaan dengan penyakit	Berhasil
10	Validasi silang penanda	`penanda C, diagnosis normal`	ValueError	ValueError: Penanda diagnosis 'C' tidak cocok dengan kata kunci diagnosis (terbaca: 'N')	Berhasil
11	Akses atribut private	`pasien.__umur`	AttributeError	AttributeError	Berhasil
12	Daftar penyakit dilindungi	`ubah list hasil getter`	Isi asli tidak berubah	jumlah penyakit tetap 0	Berhasil
13	CSV: Penanda katarak tanpa kata 'cataract'	`1,57,Male,1_left.jpg,1_right.jpg,normal fundus`	Baris dilewati + peringatan	Penanda diagnosis 'C' tidak cocok dengan kata kunci diagnosis (terbaca: 'N')	Berhasil
14	CSV: Umur berupa huruf	`2,abc,Male,2_left.jpg,2_right.jpg,normal fundu`	Baris dilewati + peringatan	Umur harus berupa bilangan bulat, bukan 'abc'	Berhasil
15	CSV: Baris kekurangan kolom	`3,40,Male`	Baris dilewati + peringatan	Nama foto mata kiri harus berakhiran _left.jpg, bukan ''	Berhasil
16	CSV: ID sama tetapi isi berbeda	`0,70,Female,0_left.jpg,0_right.jpg,cataract,no`	Baris dilewati + peringatan	Data pasien ID 0 berbeda dengan baris sebelumnya	Berhasil
17	CSV: Normal bersamaan dengan glaukoma	`5,50,Female,5_left.jpg,5_right.jpg,glaucoma,no`	Baris dilewati + peringatan	Penanda normal (N) tidak boleh bersamaan dengan penyakit	Berhasil
18	CSV: Nama foto tidak sesuai ID	`6,60,Male,9_left.jpg,6_right.jpg,normal fundus`	Baris dilewati + peringatan	Nama foto '9_left.jpg' tidak sesuai dengan ID pasien 6	Berhasil
19	CSV: Jenis kelamin tidak valid	`7,61,Laki-laki,7_left.jpg,7_right.jpg,normal f`	Baris dilewati + peringatan	Jenis kelamin harus 'Male' atau 'Female', bukan 'Laki-laki'	Berhasil
20	CSV: Diagnosis kosong	`8,62,Male,8_left.jpg,8_right.jpg,  ,normal fun`	Baris dilewati + peringatan	Diagnosis mata kiri tidak boleh kosong	Berhasil
21	CSV: Penanda bernilai 2	`9,63,Male,9_left.jpg,9_right.jpg,normal fundus`	Baris dilewati + peringatan	Penanda O harus 0 atau 1, bukan '2'	Berhasil
22	CSV tanpa kolom wajib	`header hanya ID & Patient Age`	Pesan format tidak sesuai	File CSV tidak sesuai format: Kolom 'Patient Sex' tidak ada di file CSV	Berhasil

No	Skenario	Masukan	Hasil yang Diharapkan	Hasil Pengujian	Status
23	File CSV tidak ada	`folder data kosong`	Pesan file tidak ditemukan	File CSV tidak ditemukan!	Berhasil
24	Menu 1: lihat 10 data	`pilihan 1`	10 baris data	10 baris data	Berhasil
25	Menu 2: ID ada	`ID 188`	Data lengkap + 3 kelompok	Penyakit : 3 kelompok	Berhasil
26	Menu 2: ID tidak ada	`ID 12`	Pesan tidak ditemukan	Pasien dengan ID 12 tidak ditemukan.	Berhasil
27	Menu 2: bukan angka	`"abc"`	Pesan harus angka	ID pasien harus berupa angka.	Berhasil
28	Menu 3: cari glaukoma	`pilihan 2`	206 pasien	Glaukoma: 206 pasien. 10 pertama:	Berhasil
29	Menu 4: tambah valid	`umur 63, Female, katarak & retinopati`	ID 4785 ditambahkan	Pasien ID 4785 berhasil ditambahkan (Retinopati diabetik, Katarak). Jumlah pasien sekarang: 3359	Berhasil
30	Menu 2: cari pasien baru	`ID 4785`	Ditemukan, 2 kelompok	Penyakit : 2 kelompok	Berhasil
31	Menu 4: tambah tidak valid	`jenis kelamin "Laki-laki"`	Ditolak setter	Data tidak valid: Jenis kelamin harus 'Male' atau 'Female', bukan 'Laki-laki' Pasien baru tidak ditambahkan.	Berhasil
32	Menu 5: ringkasan	`pilihan 5`	3358 pasien, terbanyak retinopati diabetik	Jumlah pasien: 3358; Retinopati diabetik (1105 pasien)	Berhasil
33	Menu: pilihan salah	`pilihan 7`	Pesan pilihan tidak tersedia	Pilihan tidak tersedia. Masukkan angka 1 sampai 6.	Berhasil
34	Menu 6: keluar	`pilihan 6`	Program selesai	Terima kasih. Program selesai.	Berhasil

3.10 Pembagian Tugas Anggota Kelompok

Pembagian tugas dalam penyusunan dan presentasi proyek ditunjukkan pada Tabel 3.8.

Tabel 3.8 Pembagian Tugas Anggota Kelompok

No	Nama	NIM	Bagian yang Dijelaskan
1	Riswan	[NIM]	Latar Belakang & Struktur Project
2	Yus Aidil	[NIM]	Dataset ODIR-5K & Pemetaan Kolom
3	Fahmi	[NIM]	Class & Object
4	Diva Nurislamia Agfa	[NIM]	Enkapsulasi
5	Rosa Ramadani	[NIM]	Inheritance: Pewarisan Bertingkat
6	Sofiang	[NIM]	Komposisi: Pemeriksaan Mata
7	Revalinda	[NIM]	Hierarki Class Penyakit & Polimorfisme

No	Nama	NIM	Bagian yang Dijelaskan
8	Andi Rezky Nabila	[NIM]	Program Utama: Membaca CSV & Menu
9	Alfira Yunita Syam	[NIM]	Hasil Program, Demo Menu & Kesimpulan

BAB IV PENUTUP

4.1 Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan hal-hal berikut.

1. Konsep *class* dan *object* berhasil diterapkan melalui sembilan *class* yang masing-masing ditempatkan pada berkas tersendiri. Program mengubah 6.392 baris data menjadi 3.358 *object* `PasienMata`, 6.716 *object* `PemeriksaanMata`, dan 7 *object* jenis penyakit.
2. Enkapsulasi diterapkan dengan atribut *private* dan 15 *setter* yang memvalidasi data dengan `raise ValueError`, termasuk validasi silang antara penanda dokter dan kata kunci diagnosis yang cocok 100% untuk 3.358 pasien.
3. *Inheritance* diterapkan pada dua hierarki: pewarisan bertingkat `Orang` → `Pasien` → `PasienMata` dan `PenyakitMata` dengan empat *class* turunan. `super().__init__()` dipakai untuk *constructor chaining*, dan *method overriding* menghasilkan perilaku polimorfis.
4. Komposisi antara `PasienMata` dan `PemeriksaanMata` memungkinkan penyakit dicatat per mata, sehingga program dapat menunjukkan apakah penyakit mengenai mata kiri, mata kanan, atau kedua mata.
5. Program membaca data nyata dengan aman: 3.034 baris milik pasien yang sama digabung, baris tidak valid dilewati dan dicatat, dan menu interaktif menyediakan fitur melihat, mencari, menambah, serta merangkum data. Penyakit terbanyak pada data adalah retinopati diabetik (1.105 pasien).

4.2 Saran

1. Menambahkan fitur ubah dan hapus data pasien agar fitur pengelolaan data (CRUD) lengkap.
2. Menyimpan data ke basis data, misalnya SQLite, agar pasien baru tersimpan secara permanen.
3. Menambahkan *class* turunan untuk kelompok penyakit lain apabila tersedia data yang lebih rinci.

4. Membuat antarmuka grafis atau berbasis web agar program lebih mudah digunakan.
5. Menambahkan pengujian otomatis (*unit test*) dan visualisasi data, misalnya grafik sebaran penyakit.

DAFTAR PUSTAKA

Larxel. (2020). *Ocular disease recognition (ODIR-5K)* [Dataset]. Kaggle. Diakses November 2026, dari https://www.kaggle.com/datasets/andrewmvd/ocular-disease-recognition-odir5k

Lott, S. F., & Phillips, D. (2021). *Python object-oriented programming: Build robust and maintainable object-oriented Python applications and libraries* (4th ed.). Packt Publishing.

Matthes, E. (2023). *Python crash course: A hands-on, project-based introduction to programming* (3rd ed.). No Starch Press.

Peking University. (2019). *International competition on ocular disease intelligent recognition (ODIR-2019)*. Diakses November 2026, dari https://odir2019.grand-challenge.org/

Python Software Foundation. (n.d.-a). *Built-in exceptions*. Python 3 Documentation. Diakses November 2026, dari https://docs.python.org/3/library/exceptions.html

Python Software Foundation. (n.d.-b). *Built-in functions*. Python 3 Documentation. Diakses November 2026, dari https://docs.python.org/3/library/functions.html

Python Software Foundation. (n.d.-c). *Classes*. The Python Tutorial. Diakses November 2026, dari https://docs.python.org/3/tutorial/classes.html

Python Software Foundation. (n.d.-d). *csv — CSV file reading and writing*. Python 3 Documentation. Diakses November 2026, dari https://docs.python.org/3/library/csv.html

World Health Organization. (2019). *World report on vision*. World Health Organization.

LAMPIRAN

Lampiran A. Kode Program Lengkap

Berikut adalah kode program lengkap. Nomor baris pada kode sama dengan nomor baris yang dirujuk pada BAB III.

A.1 orang.py

```
1  # orang.py
2  # [CLASS & OBJECT] Class Orang adalah class induk (parent class) paling atas.
3  # Atribut diambil dari kolom data: Patient Age -> umur, Patient Sex ->
   jenis_kelamin.
4
5
6  class Orang:
7      """Menyimpan data dasar seseorang: umur dan jenis kelamin."""
8
9      # Nilai jenis kelamin yang ada di data (dicek dari nilai unik kolom
   Patient Sex)
10     JENIS_KELAMIN_VALID = ("Male", "Female")
11
12     def __init__(self, umur, jenis_kelamin):
13         # [ENKAPSULASI] Nilai diisi lewat setter, jadi langsung divalidasi
14         # sebelum disimpan ke atribut private (self.__umur,
   self.__jenis_kelamin).
15         self.umur = umur
16         self.jenis_kelamin = jenis_kelamin
17
18     # ---------- [ENKAPSULASI] getter & setter ----------
19     @property
20     def umur(self):
21         return self.__umur
22
23     @umur.setter
24     def umur(self, nilai):
25         # Aturan: umur bilangan bulat 1 sampai 120 tahun
26         if not isinstance(nilai, int) or nilai < 1 or nilai > 120:
27             raise ValueError(f"Umur harus bilangan bulat 1 sampai 120, bukan
   {nilai}")
28         self.__umur = nilai
29
30     @property
31     def jenis_kelamin(self):
32         return self.__jenis_kelamin
33
34     @jenis_kelamin.setter
35     def jenis_kelamin(self, nilai):
36         # Aturan: hanya Male atau Female, sesuai isi data
37         if nilai not in Orang.JENIS_KELAMIN_VALID:
38             raise ValueError(f"Jenis kelamin harus 'Male' atau 'Female',
   bukan '{nilai}'")
39         self.__jenis_kelamin = nilai
40
41     # ---------- Method ----------
42     def tampilkan_info(self):
43         """Menampilkan data dasar. Method ini di-override oleh class
   turunan."""
44         print(f"Umur             : {self.umur} tahun")
45         print(f"Jenis Kelamin    : {self.jenis_kelamin}")
```

A.2 pasien.py

```python
# pasien.py
# [INHERITANCE] Pasien mewarisi Orang (Orang -> Pasien).
# Atribut tambahan dari kolom data: ID -> id_pasien.

from orang import Orang


class Pasien(Orang):
    """Pasien rumah sakit (turunan dari Orang)."""

    def __init__(self, id_pasien, umur, jenis_kelamin):
        # [INHERITANCE] Constructor chaining: umur & jenis kelamin diurus oleh Orang
        super().__init__(umur, jenis_kelamin)
        # [ENKAPSULASI] Diisi lewat setter agar tervalidasi
        self.id_pasien = id_pasien

    # ---------- [ENKAPSULASI] getter & setter ----------
    @property
    def id_pasien(self):
        return self.__id_pasien

    @id_pasien.setter
    def id_pasien(self, nilai):
        # Aturan: bilangan bulat 0 atau lebih (di data memang ada pasien dengan ID 0)
        if not isinstance(nilai, int) or nilai < 0:
            raise ValueError(f"ID pasien harus bilangan bulat 0 atau lebih, bukan {nilai}")
        self.__id_pasien = nilai

    # ---------- [INHERITANCE] Method overriding ----------
    def tampilkan_info(self):
        """Override tampilkan_info() milik Orang: tambah ID pasien."""
        print(f"ID Pasien        : {self.id_pasien}")
        super().tampilkan_info()  # memakai ulang tampilan umur & jenis kelamin dari Orang
```

A.3 pasien_mata.py

```python
# pasien_mata.py
# [INHERITANCE] Pewarisan bertingkat: Orang -> Pasien -> PasienMata.
# [KOMPOSISI] PasienMata MEMILIKI dua object PemeriksaanMata (mata kiri & kanan)
# dan daftar object PenyakitMata (satu pasien bisa punya lebih dari satu penyakit).
# Atribut dari kolom data: penanda N, D, G, C, A, H, M, O -> kode_diagnosis.

from pasien import Pasien
from pemeriksaan_mata import PemeriksaanMata
from penyakit_mata import PenyakitMata


class PasienMata(Pasien):
    """Pasien poli mata yang kedua matanya sudah difoto fundus."""

    KODE_PENANDA = "NDGCAHMO"  # urutan kolom penanda diagnosis di data (N = normal)

    def __init__(self, id_pasien, umur, jenis_kelamin, mata_kiri, mata_kanan,
```

```
                  kode_diagnosis, daftar_penyakit):
        # [INHERITANCE] Constructor chaining: Pasien -> Orang
        super().__init__(id_pasien, umur, jenis_kelamin)
        # [ENKAPSULASI] Urutan penting: kode_diagnosis diisi sebelum daftar_penyakit
        # karena setter daftar_penyakit mencocokkan keduanya.
        self.mata_kiri = mata_kiri
        self.mata_kanan = mata_kanan
        self.kode_diagnosis = kode_diagnosis
        self.daftar_penyakit = daftar_penyakit

    # ---------- [ENKAPSULASI] getter & setter ----------
    @property
    def mata_kiri(self):
        return self.__mata_kiri

    @mata_kiri.setter
    def mata_kiri(self, nilai):
        self.__mata_kiri = self.__cek_pemeriksaan(nilai, "kiri", "_left.jpg")

    @property
    def mata_kanan(self):
        return self.__mata_kanan

    @mata_kanan.setter
    def mata_kanan(self, nilai):
        self.__mata_kanan = self.__cek_pemeriksaan(nilai, "kanan", "_right.jpg")

    def __cek_pemeriksaan(self, nilai, sisi, akhiran):
        """Method private: pemeriksaan harus object PemeriksaanMata, sisinya benar,
        dan nama fotonya sesuai ID pasien (contoh: ID 7 -> 7_left.jpg)."""
        if not isinstance(nilai, PemeriksaanMata) or nilai.sisi != sisi:
            raise ValueError(f"Mata {sisi} harus berupa object PemeriksaanMata sisi {sisi}")
        if nilai.nama_foto != f"{self.id_pasien}{akhiran}":
            raise ValueError(f"Nama foto '{nilai.nama_foto}' tidak sesuai dengan ID pasien {self.id_pasien}")
        return nilai

    @property
    def kode_diagnosis(self):
        return self.__kode_diagnosis

    @kode_diagnosis.setter
    def kode_diagnosis(self, nilai):
        # Aturan data: minimal satu penanda, hurufnya dari N D G C A H M O,
        # tidak ada huruf ganda, dan N (normal) tidak boleh bersamaan dengan penyakit.
        if not isinstance(nilai, str) or nilai == "":
            raise ValueError("Kode diagnosis tidak boleh kosong")
        for huruf in nilai:
            if huruf not in PasienMata.KODE_PENANDA or nilai.count(huruf) > 1:
                raise ValueError(f"Kode diagnosis tidak valid: '{nilai}'")
        if "N" in nilai and len(nilai) > 1:
            raise ValueError("Penanda normal (N) tidak boleh bersamaan dengan penyakit")
        self.__kode_diagnosis = nilai

    @property
    def daftar_penyakit(self):
        return list(self.__daftar_penyakit)  # kirim salinan agar isi asli tidak diubah dari luar

```

```
    @daftar_penyakit.setter
    def daftar_penyakit(self, nilai):
        if not isinstance(nilai, list):
            raise ValueError("Daftar penyakit harus berupa list")
        kode_ditemukan = ""
        for penyakit in nilai:
            if not isinstance(penyakit, PenyakitMata):
                raise ValueError("Isi daftar penyakit harus object PenyakitMata")
            kode_ditemukan += penyakit.kode
        # Validasi silang: penyakit yang dikenali dari kata kunci diagnosis harus sama
        # dengan penanda yang diberi dokter (kolom D, G, C, A, H, M, O).
        kode_penanda = self.kode_diagnosis.replace("N", "")
        if sorted(kode_ditemukan) != sorted(kode_penanda):
            raise ValueError(f"Penanda diagnosis '{self.kode_diagnosis}' tidak cocok dengan "
                             f"kata kunci diagnosis (terbaca: '{kode_ditemukan or 'N'}')")
        self.__daftar_penyakit = list(nilai)

    # ---------- Method ----------
    def apakah_normal(self):
        """True jika pasien tidak memiliki penyakit mata."""
        return len(self.__daftar_penyakit) == 0

    def jumlah_penyakit(self):
        return len(self.__daftar_penyakit)

    def nama_penyakit(self):
        """Nama semua penyakit pasien dalam satu teks, atau 'Normal'."""
        if self.apakah_normal():
            return "Normal"
        nama = []
        for penyakit in self.__daftar_penyakit:
            nama.append(penyakit.nama)
        return ", ".join(nama)

    # ---------- [INHERITANCE] Method overriding ----------
    def tampilkan_info(self):
        """Override tampilkan_info() milik Pasien: tambah hasil pemeriksaan dan penyakit."""
        super().tampilkan_info()  # tampilkan ID, umur, jenis kelamin
        self.mata_kiri.tampilkan_info()
        self.mata_kanan.tampilkan_info()
        if self.apakah_normal():
            print("Penyakit         : tidak ada (normal)")
        else:
            print(f"Penyakit         : {self.jumlah_penyakit()} kelompok")
            for penyakit in self.__daftar_penyakit:
                # [POLIMORFISME] Versi tampilkan_info() mengikuti class object penyakitnya
                penyakit.tampilkan_info(self.mata_kiri, self.mata_kanan)
```

A.4 pemeriksaan_mata.py

```
# pemeriksaan_mata.py
# [CLASS & OBJECT] Hasil pemeriksaan foto fundus (foto retina) untuk SATU mata.
# [KOMPOSISI] Satu object PasienMata memiliki dua object PemeriksaanMata: kiri dan kanan.
# Atribut dari kolom data: Left-Fundus / Right-Fundus -> nama_foto,
# Left-Diagnostic Keywords / Right-Diagnostic Keywords -> diagnosis.

```

```python

class PemeriksaanMata:
    """Hasil foto fundus satu mata beserta kata kunci diagnosis dari dokter."""

    SISI_VALID = ("kiri", "kanan")
    # Kata kunci yang BUKAN penyakit: hasil normal dan catatan kualitas foto
    # (contoh: "lens dust" = ada debu di lensa kamera saat memotret).
    BUKAN_PENYAKIT = ("normal fundus", "lens dust", "low image quality", "image offset",
                      "no fundus image", "anterior segment image", "optic disk photographically invisible")
    KOMA_LEBAR = "\uff0c"  # tanda koma lebar (，) yang dipakai di data sebagai pemisah

    def __init__(self, sisi, nama_foto, diagnosis):
        # [ENKAPSULASI] Semua nilai lewat setter; sisi diisi lebih dulu
        # karena aturan nama_foto bergantung pada sisi mata.
        self.sisi = sisi
        self.nama_foto = nama_foto
        self.diagnosis = diagnosis

    # ---------- [ENKAPSULASI] getter & setter ----------
    @property
    def sisi(self):
        return self.__sisi

    @sisi.setter
    def sisi(self, nilai):
        if nilai not in PemeriksaanMata.SISI_VALID:
            raise ValueError(f"Sisi mata harus 'kiri' atau 'kanan', bukan '{nilai}'")
        self.__sisi = nilai

    @property
    def nama_foto(self):
        return self.__nama_foto

    @nama_foto.setter
    def nama_foto(self, nilai):
        # Aturan data: foto mata kiri berakhiran _left.jpg, mata kanan _right.jpg
        akhiran = "_left.jpg" if self.sisi == "kiri" else "_right.jpg"
        if not isinstance(nilai, str) or not nilai.endswith(akhiran):
            raise ValueError(f"Nama foto mata {self.sisi} harus berakhiran {akhiran}, bukan '{nilai}'")
        self.__nama_foto = nilai

    @property
    def diagnosis(self):
        return self.__diagnosis

    @diagnosis.setter
    def diagnosis(self, nilai):
        # Aturan: diagnosis tidak boleh kosong (spasi saja juga dianggap kosong)
        if not isinstance(nilai, str) or nilai.strip() == "":
            raise ValueError(f"Diagnosis mata {self.sisi} tidak boleh kosong")
        self.__diagnosis = nilai.strip()

    # ---------- Method ----------
    def daftar_kata_kunci(self):
        """Memecah diagnosis menjadi daftar kata kunci berhuruf kecil."""
        teks = self.diagnosis.replace(PemeriksaanMata.KOMA_LEBAR, ",").lower()
```

```
        hasil = []
        for kata in teks.split(","):
            if kata.strip() != "":
                hasil.append(kata.strip())
        return hasil

    def catatan_foto(self):
        """Catatan kualitas foto (bukan penyakit), misalnya 'lens dust'."""
        hasil = []
        for kata in self.daftar_kata_kunci():
            if kata in PemeriksaanMata.BUKAN_PENYAKIT and kata != "normal fundus":
                hasil.append(kata)
        return hasil

    def tampilkan_info(self):
        """Menampilkan diagnosis asli dari dokter untuk mata ini."""
        print(f"Mata {self.sisi:<5}       : {self.diagnosis}")
```

A.5 penyakit_mata.py

```
# penyakit_mata.py
# [CLASS & OBJECT] Class induk untuk SATU JENIS penyakit mata.
# Jenis penyakit dikenali dari kata kunci diagnosis dokter. Kode mengikuti
# kolom penanda di data: D, G, C, A, H, M, O. (Kolom N = normal, bukan penyakit.)
# [INHERITANCE] Diturunkan menjadi RetinopatiDiabetik, DegenerasiMakula,
# Glaukoma, dan KelainanLain.


class PenyakitMata:
    """Satu jenis penyakit mata beserta kata kunci pengenalnya."""

    KODE_VALID = ("D", "G", "C", "A", "H", "M", "O")

    def __init__(self, kode, nama, bagian_mata, kata_kunci):
        # [ENKAPSULASI] Semua nilai lewat setter agar tervalidasi
        self.kode = kode
        self.nama = nama
        self.bagian_mata = bagian_mata
        self.kata_kunci = kata_kunci

    # ---------- [ENKAPSULASI] getter & setter ----------
    @property
    def kode(self):
        return self.__kode

    @kode.setter
    def kode(self, nilai):
        if nilai not in PenyakitMata.KODE_VALID:
            raise ValueError(f"Kode penyakit harus salah satu dari {PenyakitMata.KODE_VALID}, bukan '{nilai}'")
        self.__kode = nilai

    @property
    def nama(self):
        return self.__nama

    @nama.setter
    def nama(self, nilai):
        if not isinstance(nilai, str) or nilai.strip() == "":
            raise ValueError("Nama penyakit tidak boleh kosong")
        self.__nama = nilai

```

```
    @property
    def bagian_mata(self):
        return self.__bagian_mata

    @bagian_mata.setter
    def bagian_mata(self, nilai):
        if not isinstance(nilai, str) or nilai.strip() == "":
            raise ValueError("Bagian mata tidak boleh kosong")
        self.__bagian_mata = nilai

    @property
    def kata_kunci(self):
        return self.__kata_kunci

    @kata_kunci.setter
    def kata_kunci(self, nilai):
        # Aturan: list berisi teks (boleh kosong untuk class turunan yang
        # mengenali penyakit dengan cara lain, misalnya KelainanLain)
        if not isinstance(nilai, list):
            raise ValueError("Kata kunci harus berupa list")
        for kata in nilai:
            if not isinstance(kata, str) or kata.strip() == "":
                raise ValueError("Setiap kata kunci harus teks yang tidak
kosong")
        self.__kata_kunci = nilai

    # ---------- Method ----------
    def ditemukan_di(self, pemeriksaan):
        """True jika salah satu kata kunci penyakit ini ada di diagnosis satu
mata."""
        for kata in pemeriksaan.daftar_kata_kunci():
            for kunci in self.kata_kunci:
                if kunci in kata:
                    return True
        return False

    def mata_terdampak(self, mata_kiri, mata_kanan):
        """Mata yang terkena penyakit ini: kedua mata, mata kiri, atau mata
kanan."""
        kiri = self.ditemukan_di(mata_kiri)
        kanan = self.ditemukan_di(mata_kanan)
        if kiri and kanan:
            return "kedua mata"
        if kiri:
            return "mata kiri"
        if kanan:
            return "mata kanan"
        return "tidak ada"

    def tampilkan_info(self, mata_kiri, mata_kanan):
        """Menampilkan nama penyakit dan mata yang terkena. Di-override class
turunan."""
        print(f"  - {self.nama} ({self.bagian_mata}):
{self.mata_terdampak(mata_kiri, mata_kanan)}")
```

A.6 retinopati_diabetik.py

```python
# retinopati_diabetik.py
# [INHERITANCE] RetinopatiDiabetik mewarisi PenyakitMata (kode D).
# Perilaku khusus: tingkat_keparahan() membaca tingkat retinopati di setiap mata
# dari kata kunci dokter (ringan, sedang, berat, proliferatif).

from penyakit_mata import PenyakitMata


class RetinopatiDiabetik(PenyakitMata):
    """Kerusakan pembuluh darah retina akibat diabetes."""

    # Urutan dari yang paling berat: (kata kunci di data, nama tingkat)
    TINGKAT = (
        ("proliferative diabetic retinopathy", "proliferatif"),
        ("severe nonproliferative retinopathy", "berat"),
        ("moderate non proliferative retinopathy", "sedang"),
        ("mild nonproliferative retinopathy", "ringan"),
    )

    def __init__(self):
        # [INHERITANCE] Constructor chaining dengan nilai khusus retinopati diabetik
        super().__init__("D", "Retinopati diabetik", "retina",
                         ["nonproliferative retinopathy", "non proliferative retinopathy",
                          "proliferative diabetic retinopathy", "diabetic retinopathy"])

    # ---------- Method khusus RetinopatiDiabetik ----------
    def tingkat_keparahan(self, pemeriksaan):
        """Tingkat retinopati diabetik di satu mata (dicari dari yang paling berat)."""
        if not self.ditemukan_di(pemeriksaan):
            return "-"
        for kunci, tingkat in RetinopatiDiabetik.TINGKAT:
            for kata in pemeriksaan.daftar_kata_kunci():
                if kunci in kata:
                    return tingkat
        return "tidak disebutkan"

    # ---------- [INHERITANCE] Method overriding ----------
    def tampilkan_info(self, mata_kiri, mata_kanan):
        """Override: tambahkan tingkat keparahan di setiap mata."""
        super().tampilkan_info(mata_kiri, mata_kanan)
        print(f"      tingkat: kiri {self.tingkat_keparahan(mata_kiri)}, "
              f"kanan {self.tingkat_keparahan(mata_kanan)}")
```

A.7 degenerasi_makula.py

```python
# degenerasi_makula.py
# [INHERITANCE] DegenerasiMakula mewarisi PenyakitMata (kode A, age-related
# macular degeneration / AMD). Perilaku khusus: jenis() membedakan AMD kering
dan basah.

from penyakit_mata import PenyakitMata


class DegenerasiMakula(PenyakitMata):
    """Kerusakan makula (pusat retina) yang berkaitan dengan usia."""

    def __init__(self):
        # [INHERITANCE] Constructor chaining dengan nilai khusus degenerasi
makula
        super().__init__("A", "Degenerasi makula", "makula (pusat retina)",
                         ["age-related macular degeneration"])

    # ---------- Method khusus DegenerasiMakula ----------
    def jenis(self, pemeriksaan):
        """Jenis AMD di satu mata: kering (dry) atau basah (wet)."""
        if not self.ditemukan_di(pemeriksaan):
            return "-"
        for kata in pemeriksaan.daftar_kata_kunci():
            if "wet age-related macular degeneration" in kata:
                return "basah"
            if "dry age-related macular degeneration" in kata:
                return "kering"
        return "tidak disebutkan"

    # ---------- [INHERITANCE] Method overriding ----------
    def tampilkan_info(self, mata_kiri, mata_kanan):
        """Override: tambahkan jenis AMD di setiap mata."""
        super().tampilkan_info(mata_kiri, mata_kanan)
        print(f"      jenis: kiri {self.jenis(mata_kiri)}, kanan
{self.jenis(mata_kanan)}")
```

A.8 glaukoma.py

```python
# glaukoma.py
# [INHERITANCE] Glaukoma mewarisi PenyakitMata (kode G).
# Perilaku khusus: status() membedakan diagnosis pasti dan dugaan ("suspected glaucoma").

from penyakit_mata import PenyakitMata


class Glaukoma(PenyakitMata):
    """Kerusakan saraf optik, sering berkaitan dengan tekanan bola mata."""

    def __init__(self):
        # [INHERITANCE] Constructor chaining dengan nilai khusus glaukoma
        super().__init__("G", "Glaukoma", "saraf optik", ["glaucoma"])

    # ---------- Method khusus Glaukoma ----------
    def status(self, pemeriksaan):
        """Status glaukoma di satu mata: pasti, dugaan, atau '-' jika tidak ada."""
        hasil = "-"
        for kata in pemeriksaan.daftar_kata_kunci():
            if "glaucoma" in kata:
                if "suspected" in kata and hasil == "-":
                    hasil = "dugaan"
                elif "suspected" not in kata:
                    hasil = "pasti"
        return hasil

    # ---------- [INHERITANCE] Method overriding ----------
    def tampilkan_info(self, mata_kiri, mata_kanan):
        """Override: tambahkan status diagnosis di setiap mata."""
        super().tampilkan_info(mata_kiri, mata_kanan)
        print(f"      status: kiri {self.status(mata_kiri)}, kanan {self.status(mata_kanan)}")
```

A.9 kelainan_lain.py

```python
# kelainan_lain.py
# [INHERITANCE] KelainanLain mewarisi PenyakitMata (kode O = other).
# Kelainan lain tidak punya daftar kata kunci tetap (ada puluhan jenis, misalnya
# drusen dan membran epiretina), jadi cara mengenalinya BERBEDA dari induknya:
# kata kunci yang bukan hasil normal, bukan catatan foto, dan bukan milik
# 6 kelompok penyakit lain dianggap kelainan lain.
# [POLIMORFISME] Method ditemukan_di() di-override.

from penyakit_mata import PenyakitMata
from pemeriksaan_mata import PemeriksaanMata


class KelainanLain(PenyakitMata):
    """Kelainan mata di luar 6 kelompok penyakit utama."""

    def __init__(self, kata_kunci_kelompok_lain):
        # [INHERITANCE] Kata kunci induk dikosongkan karena cara mengenalinya berbeda
        super().__init__("O", "Kelainan lain", "bervariasi", [])
        self.kata_kunci_kelompok_lain = kata_kunci_kelompok_lain

    # ---------- [ENKAPSULASI] getter & setter ----------
```

```
    @property
    def kata_kunci_kelompok_lain(self):
        return self.__kata_kunci_kelompok_lain

    @kata_kunci_kelompok_lain.setter
    def kata_kunci_kelompok_lain(self, nilai):
        if not isinstance(nilai, list) or len(nilai) == 0:
            raise ValueError("Kata kunci kelompok penyakit lain harus list yang tidak kosong")
        self.__kata_kunci_kelompok_lain = nilai

    # ---------- Method khusus KelainanLain ----------
    def daftar_kelainan(self, pemeriksaan):
        """Kata kunci di satu mata yang termasuk kelainan lain."""
        hasil = []
        for kata in pemeriksaan.daftar_kata_kunci():
            if kata in PemeriksaanMata.BUKAN_PENYAKIT:
                continue
            milik_kelompok_lain = False
            for kunci in self.kata_kunci_kelompok_lain:
                if kunci in kata:
                    milik_kelompok_lain = True
            if not milik_kelompok_lain:
                hasil.append(kata)
        return hasil

    # ---------- [POLIMORFISME] Method overriding ----------
    def ditemukan_di(self, pemeriksaan):
        """Override: kelainan lain ditemukan jika daftar_kelainan() tidak kosong."""
        return len(self.daftar_kelainan(pemeriksaan)) > 0

    def tampilkan_info(self, mata_kiri, mata_kanan):
        """Override: tambahkan jenis kelainan yang ditemukan."""
        super().tampilkan_info(mata_kiri, mata_kanan)
        semua = []
        for kata in self.daftar_kelainan(mata_kiri) + self.daftar_kelainan(mata_kanan):
            if kata not in semua:
                semua.append(kata)
        print(f"      jenis: {', '.join(semua)}")
```

A.10 main.py

```
# main.py
# Program utama Sistem Manajemen Data Pasien Penyakit Mata.
# Cara menjalankan (dari dalam folder program): python main.py
# Hanya memakai library bawaan Python (csv dan os).
# Data: ODIR-5K (Ocular Disease Intelligent Recognition), file data/full_df.csv.

import csv
import os

from degenerasi_makula import DegenerasiMakula
from glaukoma import Glaukoma
from kelainan_lain import KelainanLain
from pasien_mata import PasienMata
from pemeriksaan_mata import PemeriksaanMata
from penyakit_mata import PenyakitMata
from retinopati_diabetik import RetinopatiDiabetik

# File CSV dicari di folder data/ (di samping main.py) atau di folder yang sama dengan main.py.
```

```
FOLDER_PROGRAM = os.path.dirname(os.path.abspath(__file__))
LOKASI_DATA = [os.path.join(FOLDER_PROGRAM, "data", "full_df.csv"),
               os.path.join(FOLDER_PROGRAM, "full_df.csv")]
JUMLAH_DITAMPILKAN = 10

# Kolom yang berisi data pasien. Satu pasien bisa muncul di 2 baris CSV
# (satu baris per foto mata), dan isi kolom-kolom ini harus sama di kedua baris.
KOLOM_PASIEN = ["ID", "Patient Age", "Patient Sex", "Left-Fundus", "Right-Fundus",
                "Left-Diagnostic Keywords", "Right-Diagnostic Keywords",
                "N", "D", "G", "C", "A", "H", "M", "O"]


def cetak_judul(judul):
    print()
    print("=" * 64)
    print(judul)
    print("=" * 64)


def cari_file_csv():
    """Mengembalikan lokasi file CSV, atau None jika tidak ditemukan."""
    for lokasi in LOKASI_DATA:
        if os.path.exists(lokasi):
            return lokasi
    return None


def ke_bilangan_bulat(teks, nama_kolom):
    """Mengubah teks menjadi bilangan bulat, dengan pesan error berbahasa Indonesia."""
    try:
        return int(teks)
    except ValueError:
        raise ValueError(f"{nama_kolom} harus berupa bilangan bulat, bukan '{teks.strip()}'")


def buat_daftar_jenis_penyakit():
    """[CLASS & OBJECT] Membuat 7 object jenis penyakit mata (satu per kolom penanda)."""
    retinopati = RetinopatiDiabetik()
    glaukoma = Glaukoma()
    katarak = PenyakitMata("C", "Katarak", "lensa mata", ["cataract"])
    makula = DegenerasiMakula()
    hipertensi = PenyakitMata("H", "Retinopati hipertensi", "retina", ["hypertensive retinopathy"])
    miopia = PenyakitMata("M", "Miopia patologis", "retina",
                          ["pathological myopia", "myopia retinopathy", "myopic maculopathy"])
    enam_kelompok = [retinopati, glaukoma, katarak, makula, hipertensi, miopia]
    # KelainanLain perlu tahu kata kunci 6 kelompok lain agar tidak terhitung dua kali
    kata_kunci_lain = []
    for penyakit in enam_kelompok:
        kata_kunci_lain.extend(penyakit.kata_kunci)
    return enam_kelompok + [KelainanLain(kata_kunci_lain)]


def baca_kode_diagnosis(baris):
    """Menggabungkan penanda N, D, G, C, A, H, M, O yang bernilai 1, misalnya 'DO'."""
    kode = ""
    for huruf in PasienMata.KODE_PENANDA:
```

```
        nilai = baris[huruf].strip()
        if nilai not in ("0", "1"):
            raise ValueError(f"Penanda {huruf} harus 0 atau 1, bukan '{nilai}'")
        if nilai == "1":
            kode += huruf
    return kode


def deteksi_penyakit(mata_kiri, mata_kanan, daftar_jenis):
    """Mencari jenis penyakit yang kata kuncinya ada di mata kiri atau kanan."""
    hasil = []
    for penyakit in daftar_jenis:
        # [POLIMORFISME] ditemukan_di() milik KelainanLain bekerja berbeda dari induknya
        if penyakit.ditemukan_di(mata_kiri) or penyakit.ditemukan_di(mata_kanan):
            hasil.append(penyakit)
    return hasil


def buat_pasien(baris, daftar_jenis):
    """[CLASS & OBJECT] Mengubah satu baris CSV menjadi object PasienMata."""
    # [KOMPOSISI] Object PemeriksaanMata dibuat dulu, lalu dimasukkan ke PasienMata
    mata_kiri = PemeriksaanMata("kiri", baris["Left-Fundus"].strip(), baris["Left-Diagnostic Keywords"])
    mata_kanan = PemeriksaanMata("kanan", baris["Right-Fundus"].strip(), baris["Right-Diagnostic Keywords"])
    return PasienMata(
        ke_bilangan_bulat(baris["ID"], "ID"),
        ke_bilangan_bulat(baris["Patient Age"], "Umur"),
        baris["Patient Sex"].strip(),
        mata_kiri,
        mata_kanan,
        baca_kode_diagnosis(baris),
        deteksi_penyakit(mata_kiri, mata_kanan, daftar_jenis),
    )


def muat_data(lokasi_file, daftar_jenis):
    """Membaca CSV, menggabungkan baris milik pasien yang sama, dan mencatat baris tidak valid."""
    daftar_pasien = []
    isi_per_id = {}        # ID -> isi kolom pasien dari baris pertama yang valid
    baris_dilewati = []    # nomor baris CSV yang tidak valid
    baris_digabung = 0     # baris foto mata kedua dari pasien yang sama
    # utf-8-sig: tetap aman jika CSV pernah disimpan ulang lewat Excel (ada BOM)
    with open(lokasi_file, mode="r", encoding="utf-8-sig", newline="") as file:
        # restval="": kolom yang hilang dianggap teks kosong, jadi ikut ditolak validasi
        pembaca = csv.DictReader(file, restval="")
        for kolom in KOLOM_PASIEN:
            if kolom not in (pembaca.fieldnames or []):
                raise ValueError(f"Kolom '{kolom}' tidak ada di file CSV")
        for baris in pembaca:
            try:
                isi = tuple(baris[kolom] for kolom in KOLOM_PASIEN)
                id_teks = baris["ID"].strip()
                if id_teks in isi_per_id:
                    # Baris foto mata kedua: isinya harus sama dengan baris pertama
```

```
                    if isi != isi_per_id[id_teks]:
                        raise ValueError(f"Data pasien ID {id_teks} berbeda dengan baris sebelumnya")
                    baris_digabung += 1
                    continue
                daftar_pasien.append(buat_pasien(baris, daftar_jenis))
                isi_per_id[id_teks] = isi
            except ValueError as error:
                # Baris yang melanggar aturan dilewati dan dicatat, program tetap berjalan
                baris_dilewati.append(pembaca.line_num)
                print(f"[PERINGATAN] Baris {pembaca.line_num} dilewati: {error}")
    return daftar_pasien, baris_dilewati, baris_digabung


def demo_validasi(daftar_pasien):
    """[ENKAPSULASI] Contoh data tidak valid yang ditolak setter."""
    cetak_judul("DEMO ENKAPSULASI (data tidak valid ditolak)")
    pasien = daftar_pasien[0]
    print(f"Umur pasien ID {pasien.id_pasien} sebelum diubah : {pasien.umur}")
    print("Mencoba mengubah: pasien.umur = -5")
    try:
        pasien.umur = -5  # tidak valid, umur harus 1 sampai 120
    except ValueError as error:
        print(f"ValueError tertangkap: {error}")
    print(f"Umur pasien ID {pasien.id_pasien} sesudahnya     : {pasien.umur} (tidak berubah)")
    print()
    print("Mencoba membuat pasien berpenanda katarak (C), tetapi diagnosisnya normal")
    try:
        mata_kiri = PemeriksaanMata("kiri", "9999_left.jpg", "normal fundus")
        mata_kanan = PemeriksaanMata("kanan", "9999_right.jpg", "normal fundus")
        PasienMata(9999, 50, "Male", mata_kiri, mata_kanan, "C", [])
    except ValueError as error:
        print(f"ValueError tertangkap: {error}")


# ======================== MENU INTERAKTIF ========================

def menu_lihat_data(daftar_pasien):
    """Menu 1: menampilkan 10 pasien pertama dalam tabel singkat."""
    cetak_judul(f"{JUMLAH_DITAMPILKAN} DATA PERTAMA (dari {len(daftar_pasien)} pasien)")
    print(f"{'ID':<6}{'Umur':<6}{'Kelamin':<9}Penyakit")
    for pasien in daftar_pasien[:JUMLAH_DITAMPILKAN]:
        print(f"{pasien.id_pasien:<6}{pasien.umur:<6}{pasien.jenis_kelamin:<9}{pasien.nama_penyakit()}")


def cari_pasien(daftar_pasien, id_pasien):
    """Mengembalikan pasien dengan ID tersebut, atau None jika tidak ada."""
    for pasien in daftar_pasien:
        if pasien.id_pasien == id_pasien:
            return pasien
    return None


def menu_cari_id(daftar_pasien):
    """Menu 2: mencari pasien berdasarkan ID."""
    cetak_judul("CARI PASIEN BERDASARKAN ID")
    try:
```

```
        id_pasien = int(input("Masukkan ID pasien: "))
    except ValueError:
        print("ID pasien harus berupa angka.")
        return
    pasien = cari_pasien(daftar_pasien, id_pasien)
    if pasien is None:
        print(f"Pasien dengan ID {id_pasien} tidak ditemukan.")
    else:
        pasien.tampilkan_info()


def menu_cari_penyakit(daftar_pasien, daftar_jenis):
    """Menu 3: menampilkan pasien yang memiliki penyakit tertentu."""
    cetak_judul("CARI PASIEN BERDASARKAN PENYAKIT")
    for nomor, penyakit in enumerate(daftar_jenis, start=1):
        print(f"{nomor}. {penyakit.nama}")
    try:
        pilihan = int(input(f"Pilih penyakit (1-{len(daftar_jenis)}): "))
    except ValueError:
        print("Pilihan harus berupa angka.")
        return
    if pilihan < 1 or pilihan > len(daftar_jenis):
        print("Pilihan tidak tersedia.")
        return
    penyakit = daftar_jenis[pilihan - 1]
    hasil = []
    for pasien in daftar_pasien:
        if penyakit in pasien.daftar_penyakit:
            hasil.append(pasien)
    print(f"\n{penyakit.nama}: {len(hasil)} pasien. {JUMLAH_DITAMPILKAN}
pertama:")
    print(f"{'ID':<6}{'Umur':<6}{'Kelamin':<9}Mata terkena")
    for pasien in hasil[:JUMLAH_DITAMPILKAN]:
        mata = penyakit.mata_terdampak(pasien.mata_kiri, pasien.mata_kanan)

print(f"{pasien.id_pasien:<6}{pasien.umur:<6}{pasien.jenis_kelamin:<9}{mata}"
)


def menu_tambah_pasien(daftar_pasien, daftar_jenis):
    """Menu 4: menambah pasien baru ke list (file CSV asli tidak diubah)."""
    cetak_judul("TAMBAH PASIEN BARU")
    # ID dibuat otomatis (ID terbesar + 1) agar tidak ada ID ganda
    id_baru = max(pasien.id_pasien for pasien in daftar_pasien) + 1
    print(f"ID pasien baru (otomatis)    : {id_baru}")
    print("Tulis diagnosis dengan istilah dokter seperti di data, pisahkan
dengan koma.")
    print("Contoh: normal fundus / cataract / glaucoma / mild
nonproliferative retinopathy")
    try:
        umur = ke_bilangan_bulat(input("Umur                          : "),
"Umur")
        jenis_kelamin = input("Jenis kelamin (Male/Female)  : ").strip()
        mata_kiri = PemeriksaanMata("kiri", f"{id_baru}_left.jpg",
input("Diagnosis mata kiri          : "))
        mata_kanan = PemeriksaanMata("kanan", f"{id_baru}_right.jpg",
input("Diagnosis mata kanan         : "))
        daftar_penyakit = deteksi_penyakit(mata_kiri, mata_kanan,
daftar_jenis)
        kode = ""
        for penyakit in daftar_penyakit:
            kode += penyakit.kode
        # [ENKAPSULASI] Data baru diperiksa oleh setter yang sama dengan data
CSV
        pasien = PasienMata(id_baru, umur, jenis_kelamin, mata_kiri,
mata_kanan, kode or "N", daftar_penyakit)
```

```
    except ValueError as error:
        print(f"Data tidak valid: {error}")
        print("Pasien baru tidak ditambahkan.")
        return
    daftar_pasien.append(pasien)
    print(f"Pasien ID {id_baru} berhasil ditambahkan ({pasien.nama_penyakit()}). "
          f"Jumlah pasien sekarang: {len(daftar_pasien)}")


def menu_ringkasan(daftar_pasien, daftar_jenis):
    """Menu 5: ringkasan jumlah pasien, umur, dan setiap penyakit."""
    cetak_judul("RINGKASAN DATA")
    total = len(daftar_pasien)
    laki_laki = 0
    total_umur = 0
    normal = 0
    lebih_dari_satu = 0
    for pasien in daftar_pasien:
        total_umur += pasien.umur
        if pasien.jenis_kelamin == "Male":
            laki_laki += 1
        if pasien.apakah_normal():
            normal += 1
        if pasien.jumlah_penyakit() > 1:
            lebih_dari_satu += 1
    print(f"Jumlah pasien             : {total}")
    print(f"Laki-laki / Perempuan     : {laki_laki} / {total - laki_laki}")
    print(f"Rata-rata umur            : {total_umur / total:.1f} tahun")
    print(f"Pasien normal             : {normal}")
    print(f"Pasien > 1 kelompok       : {lebih_dari_satu}")
    print()
    print(f"{'Penyakit':<24}{'Pasien':>7}{'Rata umur':>11}{'Kedua mata':>12}")
    terbanyak = None
    jumlah_terbanyak = 0
    for penyakit in daftar_jenis:
        jumlah = 0
        total_umur_penyakit = 0
        kedua_mata = 0
        for pasien in daftar_pasien:
            if penyakit in pasien.daftar_penyakit:
                jumlah += 1
                total_umur_penyakit += pasien.umur
                if penyakit.mata_terdampak(pasien.mata_kiri, pasien.mata_kanan) == "kedua mata":
                    kedua_mata += 1
        rata_umur = total_umur_penyakit / jumlah if jumlah > 0 else 0

        print(f"{penyakit.nama:<24}{jumlah:>7}{rata_umur:>11.1f}{kedua_mata:>12}")
        if jumlah > jumlah_terbanyak:
            terbanyak = penyakit
            jumlah_terbanyak = jumlah
    if terbanyak is not None:
        print(f"\nPenyakit terbanyak        : {terbanyak.nama} ({jumlah_terbanyak} pasien)")


def jalankan_menu(daftar_pasien, daftar_jenis):
    """Menampilkan menu berulang-ulang sampai pengguna memilih Keluar."""
    while True:
        cetak_judul("MENU")
        print(f"1. Lihat {JUMLAH_DITAMPILKAN} data pertama")
        print("2. Cari pasien berdasarkan ID")
        print("3. Cari pasien berdasarkan penyakit")
        print("4. Tambah pasien baru")
```

```
        print("5. Tampilkan ringkasan")
        print("6. Keluar")
        pilihan = input("Pilih menu (1-6): ").strip()
        if pilihan == "1":
            menu_lihat_data(daftar_pasien)
        elif pilihan == "2":
            menu_cari_id(daftar_pasien)
        elif pilihan == "3":
            menu_cari_penyakit(daftar_pasien, daftar_jenis)
        elif pilihan == "4":
            menu_tambah_pasien(daftar_pasien, daftar_jenis)
        elif pilihan == "5":
            menu_ringkasan(daftar_pasien, daftar_jenis)
        elif pilihan == "6":
            print("Terima kasih. Program selesai.")
            break
        else:
            print("Pilihan tidak tersedia. Masukkan angka 1 sampai 6.")


def main():
    cetak_judul("SISTEM MANAJEMEN DATA PASIEN PENYAKIT MATA")

    lokasi_file = cari_file_csv()
    if lokasi_file is None:
        print("File CSV tidak ditemukan!")
        print("Letakkan 'full_df.csv' di folder data/ di samping main.py.")
        return

    # [CLASS & OBJECT] 7 object jenis penyakit, lalu setiap pasien menjadi
object PasienMata
    daftar_jenis = buat_daftar_jenis_penyakit()
    try:
        daftar_pasien, baris_dilewati, baris_digabung =
muat_data(lokasi_file, daftar_jenis)
    except ValueError as error:
        print(f"File CSV tidak sesuai format: {error}")
        return
    print(f"File data          : {os.path.basename(lokasi_file)}")
    print(f"Pasien dimuat      : {len(daftar_pasien)} object PasienMata")
    print(f"Baris digabung     : {baris_digabung} (foto mata kedua dari
pasien yang sama)")
    print(f"Baris dilewati     : {len(baris_dilewati)}")
    if len(baris_dilewati) > 0:
        print(f"Nomor baris CSV    : {baris_dilewati}")

    if len(daftar_pasien) == 0:
        print("Tidak ada data valid yang bisa diproses.")
        return

    demo_validasi(daftar_pasien)
    jalankan_menu(daftar_pasien, daftar_jenis)


if __name__ == "__main__":
    main()
```

Lampiran B. Keluaran Lengkap Program

Keluaran lengkap saat program dijalankan dengan perintah `python main.py`, termasuk satu sesi menu interaktif. Masukan yang diketik pada sesi tersebut: pilihan 1; pilihan 2 dengan ID 188, 12, dan `abc`; pilihan 3 dengan penyakit

nomor 2 (glaukoma); pilihan 5; pilihan 4 dengan data pasien baru; pilihan 2 dengan ID 4785; pilihan 4 dengan jenis kelamin “Laki-laki”; pilihan 7; lalu pilihan 6 untuk keluar.

```
> python main.py
================================================================
SISTEM MANAJEMEN DATA PASIEN PENYAKIT MATA
================================================================
File data          : full_df.csv
Pasien dimuat      : 3358 object PasienMata
Baris digabung     : 3034 (foto mata kedua dari pasien yang sama)
Baris dilewati     : 0

================================================================
DEMO ENKAPSULASI (data tidak valid ditolak)
================================================================
Umur pasien ID 0 sebelum diubah : 69
Mencoba mengubah: pasien.umur = -5
ValueError tertangkap: Umur harus bilangan bulat 1 sampai 120, bukan -5
Umur pasien ID 0 sesudahnya     : 69 (tidak berubah)

Mencoba membuat pasien berpenanda katarak (C), tetapi diagnosisnya normal
ValueError tertangkap: Penanda diagnosis 'C' tidak cocok dengan kata kunci
diagnosis (terbaca: 'N')

================================================================
MENU
================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 1

================================================================
10 DATA PERTAMA (dari 3358 pasien)
================================================================
ID    Umur  Kelamin  Penyakit
0     69    Female   Katarak
1     57    Male     Normal
2     42    Male     Retinopati diabetik, Kelainan lain
4     53    Male     Retinopati diabetik, Kelainan lain
5     50    Female   Retinopati diabetik
6     60    Male     Retinopati diabetik, Kelainan lain
7     60    Female   Retinopati diabetik, Kelainan lain
8     59    Male     Normal
9     54    Male     Kelainan lain
10    70    Male     Kelainan lain

================================================================
MENU
================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 2

================================================================
CARI PASIEN BERDASARKAN ID
```

```
================================================================
Masukkan ID pasien: 188
ID Pasien        : 188
Umur             : 55 tahun
Jenis Kelamin    : Female
Mata kiri        : laser spot, severe proliferative diabetic retinopathy
Mata kanan       : cataract
Penyakit         : 3 kelompok
  - Retinopati diabetik (retina): mata kiri
      tingkat: kiri proliferatif, kanan -
  - Katarak (lensa mata): mata kanan
  - Kelainan lain (bervariasi): mata kiri
      jenis: laser spot

================================================================
MENU
================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 2

================================================================
CARI PASIEN BERDASARKAN ID
================================================================
Masukkan ID pasien: 12
Pasien dengan ID 12 tidak ditemukan.

================================================================
MENU
================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 2

================================================================
CARI PASIEN BERDASARKAN ID
================================================================
Masukkan ID pasien: abc
ID pasien harus berupa angka.

================================================================
MENU
================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 3

================================================================
CARI PASIEN BERDASARKAN PENYAKIT
================================================================
1. Retinopati diabetik
2. Glaukoma
3. Katarak
4. Degenerasi makula
```

```
5. Retinopati hipertensi
6. Miopia patologis
7. Kelainan lain
Pilih penyakit (1-7): 2

Glaukoma: 206 pasien. 10 pertama:
ID    Umur  Kelamin  Mata terkena
43    35    Male     mata kanan
95    46    Male     mata kiri
153   79    Male     mata kiri
167   71    Male     kedua mata
178   54    Male     mata kiri
238   70    Male     mata kanan
247   49    Male     kedua mata
365   60    Female   kedua mata
583   54    Male     kedua mata
625   72    Female   mata kiri

================================================================
MENU
================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 5

================================================================
RINGKASAN DATA
================================================================
Jumlah pasien             : 3358
Laki-laki / Perempuan     : 1800 / 1558
Rata-rata umur            : 57.9 tahun
Pasien normal             : 1080
Pasien > 1 kelompok       : 556

Penyakit                Pasien  Rata umur  Kedua mata
Retinopati diabetik       1105       56.4         659
Glaukoma                   206       62.6         110
Katarak                    208       66.4          99
Degenerasi makula          163       60.9         116
Retinopati hipertensi      103       56.2          90
Miopia patologis           171       57.0          91
Kelainan lain              905       59.1         173

Penyakit terbanyak        : Retinopati diabetik (1105 pasien)

================================================================
MENU
================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 4

================================================================
TAMBAH PASIEN BARU
================================================================
ID pasien baru (otomatis)    : 4785
Tulis diagnosis dengan istilah dokter seperti di data, pisahkan dengan koma.
Contoh: normal fundus / cataract / glaucoma / mild nonproliferative retinopathy
```

```
Umur                         : 63
Jenis kelamin (Male/Female)  : Female
Diagnosis mata kiri          : cataract
Diagnosis mata kanan         : moderate non proliferative retinopathy
Pasien ID 4785 berhasil ditambahkan (Retinopati diabetik, Katarak). Jumlah pasien
sekarang: 3359

==================================================================
MENU
==================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 2

==================================================================
CARI PASIEN BERDASARKAN ID
==================================================================
Masukkan ID pasien: 4785
ID Pasien        : 4785
Umur             : 63 tahun
Jenis Kelamin    : Female
Mata kiri        : cataract
Mata kanan       : moderate non proliferative retinopathy
Penyakit         : 2 kelompok
  - Retinopati diabetik (retina): mata kanan
      tingkat: kiri -, kanan sedang
  - Katarak (lensa mata): mata kiri

==================================================================
MENU
==================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 4

==================================================================
TAMBAH PASIEN BARU
==================================================================
ID pasien baru (otomatis)    : 4786
Tulis diagnosis dengan istilah dokter seperti di data, pisahkan dengan koma.
Contoh: normal fundus / cataract / glaucoma / mild nonproliferative retinopathy
Umur                         : 45
Jenis kelamin (Male/Female)  : Laki-laki
Diagnosis mata kiri          : normal fundus
Diagnosis mata kanan         : normal fundus
Data tidak valid: Jenis kelamin harus 'Male' atau 'Female', bukan 'Laki-laki'
Pasien baru tidak ditambahkan.

==================================================================
MENU
==================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 7
```

```
Pilihan tidak tersedia. Masukkan angka 1 sampai 6.

==================================================================
MENU
==================================================================
1. Lihat 10 data pertama
2. Cari pasien berdasarkan ID
3. Cari pasien berdasarkan penyakit
4. Tambah pasien baru
5. Tampilkan ringkasan
6. Keluar
Pilih menu (1-6): 6
Terima kasih. Program selesai.
```